

IC Validator

Generic ESD Library

Overview

SYNOPSYS®

Contents

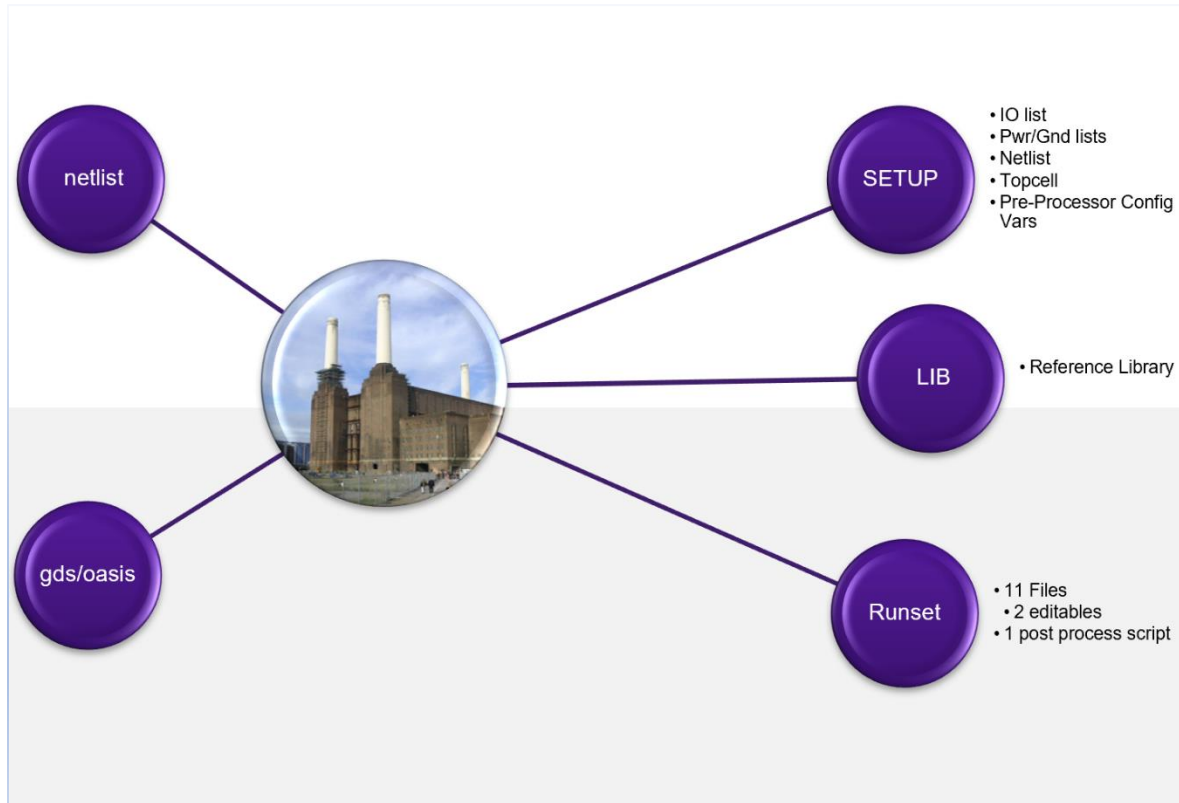
Contents	2
ICV PERC ESD Library and Generic PDK.....	4
Overview	4
Document Color Key	7
IC Validator Generic ESD Testcase	7
Runset File Overview	7
Runset Files	9
esd_core.rs.....	9
esd_core_init.rs	10
esd_devices.rs.....	10
esd_violation_defs.rs.....	10
esd_topology_models.py.....	10
esd_preprocess.py	11
esd_rules.py	11
pres_violation_defs.py.....	11
pres_rules.py.....	12
esd_utility.py.....	12
Topology Check Command Execution Flow	13
esd_setup.rs.....	13
esd_core.rs.....	13
Current Density Check Command Execution Flow	13
esd_core.py.....	14
esd_core.rs (Return)	14
Running IC Validator PERC & Exercises	16
Testcase Contents	16
Running IC Validator PERC	17
Netlist-Only Checks	17
All Checks Including Current Density and Point-to-point Resistance	17
PERC Topology Checking Exercises	18
Setup	18
Exercise 1 - Defining Devices Available for ESD Protection	18
Exercise 2 - Defining ESD Structure Details.....	19

Exercise 3 - Adding Rules / Checking Device Properties	21
PERC Current Density and Point-to-Point Resistance Checking Exercise	23
Setup	23
Exercise 4 - Defining Current Density and Point-to-point Resistance Paths.....	23
ESD Library Classes and Functions	31
SeriesComposite	31
Class Methods.....	31
PathComposite.....	32
Class Methods.....	32
SeriesCompositeDef.....	37
Class Methods.....	37
PathCompositeDef	40
Class Methods.....	40
TopologyDefDB	43
Class Methods.....	43
SeriesCompositeTable.....	44
Class Methods.....	44
General Library Functions	51
ESD Rules and Diagrams	52

ICV PERC ESD Library and Generic PDK

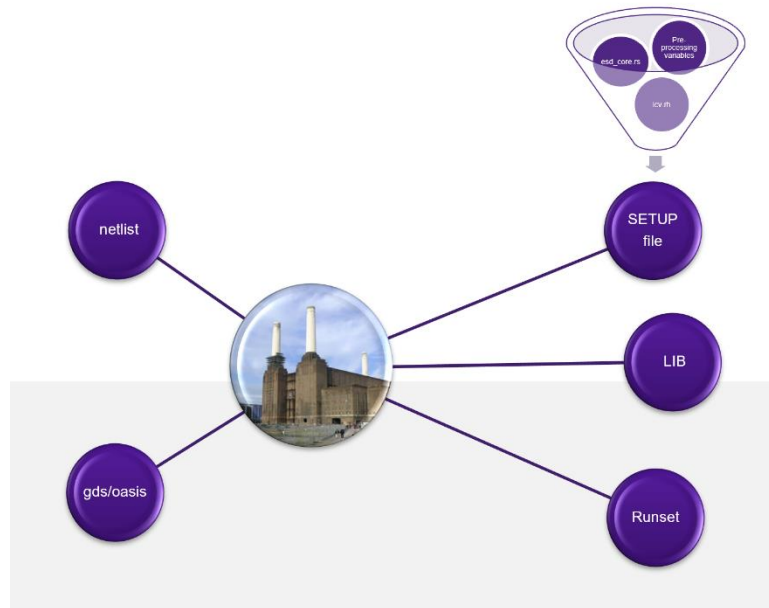
Overview

The ICV PERC product includes a “reference flow” for ESD topology and layout verification. This is commonly called the ICV PERC ESD Library. This document provides a high-level overview of the ICV PERC ESD Library and provides information to enable the user to execute and modify the rules for a generic testcase.

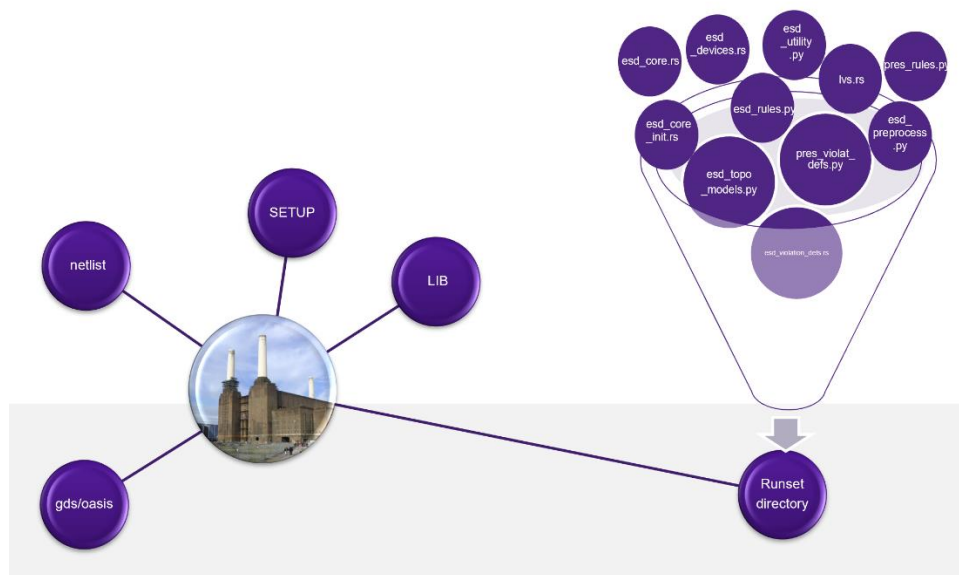


The main idea of the ESD Library is that the topology for protection structures consists of Primary and Secondary circuitry, Power/Ground Clamps, and anti-parallel diode clamps. A reference flow is useful to enable easy migration of ESD rules to different process nodes that may use different device types in the specific topologies. The picture above shows the different inputs to the factory or engine and demonstrates how the rules are organized into different areas. A product user for an enabled process technology will only need to modify data in the SETUP area. A developer will need to modify data in both the SETUP and Runset areas. Data in the LIBRARY area will not need to be modified.

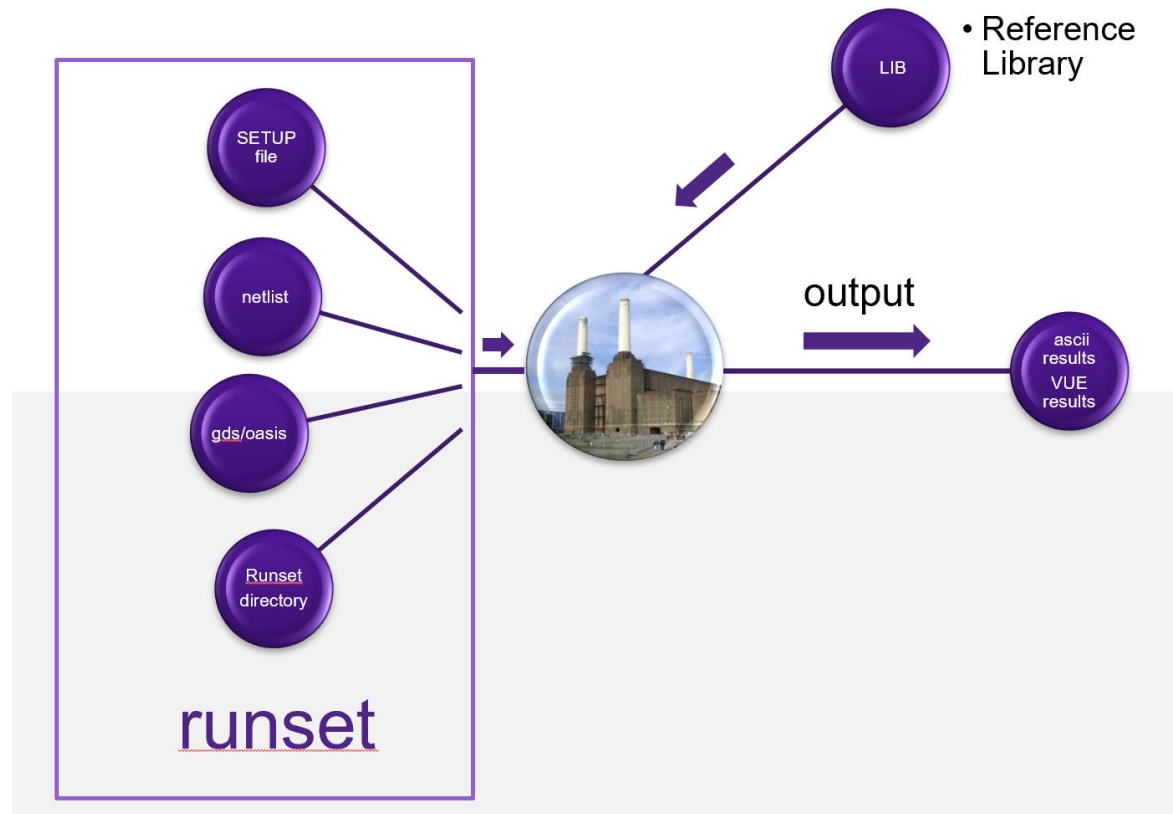
The setup file contains user input for netlist or layout input paths and topcell name, environment variable settings, and include statements to required files. This can be seen in the figure below:



For the runset developer, the files found in the RUNSET directory are required to be modified to support different layer maps, unique devices, different model names, and process specific rules. This can be seen in the following picture:



The files, environment variables, RUNSET directory, and input data are commonly referred to as a “runset”. This runset is used to execute ICV PERC and generate both ascii reports and data for the VUE tool that enables cross probing capability into layout and schematic tools.



Document Color Key

This document refers to different functions that are called in the example ICV PERC Generic ESD Testcase. This document will use the below colors to differentiate functions that are in the ICV Reference manual, Synopsys ESD Library, or functions in the runset files themselves.

Green – Functions that can be found in the ICV reference manual

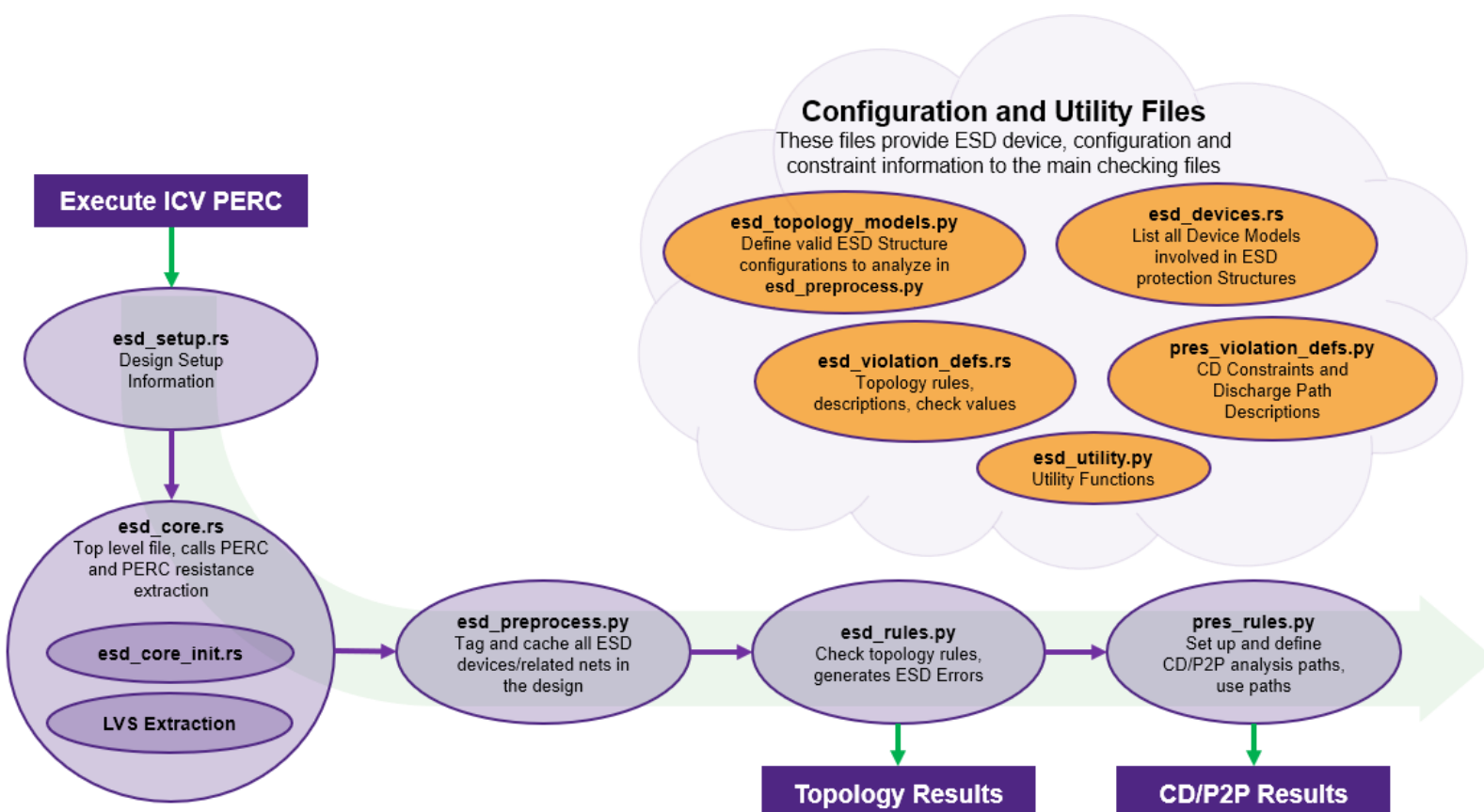
Orange – ESD Library functions or variables

Purple – Functions found in the runset

IC Validator Generic ESD Testcase

Runset File Overview

The below picture shows the flow of the ICV files called in the generic ESD testcase. The files along the green arrow are files which process PERC commands, check topologies and report errors. The files inside the cloud are configuration and utility files. These files will likely change from technology node to different technology nodes. The main files however will likely change little from node to node.

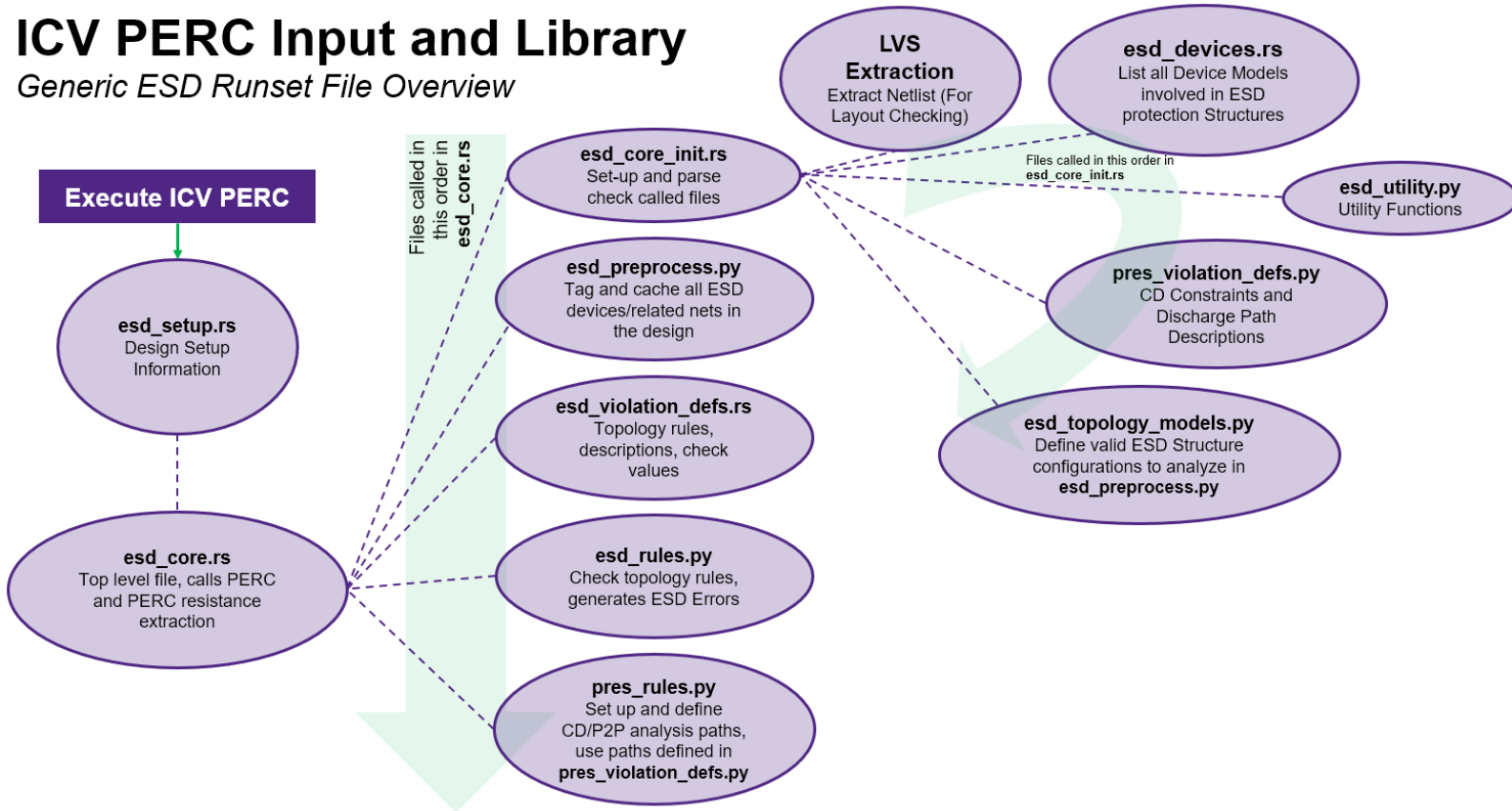


Below is an outline that provides an overview of the files used in the Generic ESD Runset. The files are included in the order shown in the outline (and are described in more detail in later sections of this document. For quick reference a picture is included after the outline that illustrates where the files are called.

1. **esd_setup.rs** – Design Setup and Information (Paths to GDS/OASIS, paths to runset files, etc.)
2. **esd_core.rs** – Top level file, calls PERC and PERC resistance extraction. Calls the below files in the below order
 - a. **esd_core_init.rs** – Set up and parse files that define rules and valid protection structures. Calls the below files in the below order.
 - i. **LVS Extraction Deck** – Extract layout netlist (Layout based checking only)
 - ii. **esd_devices.rs** – Lists all valid device models involved in ESD checking and protection structures.
 - iii. **esd_utility.py** – File that has utility functions for checking (Likely will not need to be modified).
 - iv. **pres_violation_defs.py** – Contains CD constraints and discharge path descriptions as well as tags for the CD/P2P checking rules.
 - v. **esd_topology_models.py** – Define valid ESD structures that will be checked
 - b. **esd_preprocess.py** – Tag and cache valid nets (power, ground, I/O nets) and ESD devices. Populate structures (Defined in the ESD_Library) that organize ESD information so that it can be used to more easily analyze ESD structures and CD/P2P analysis paths.
 - c. **esd_rules.py** – Check Topology rules and output ESD errors
 - d. **pres_rules.py** – Set up and define CD/P2P paths for analysis. Use path tags defined in the pres_violation_defs.py file

ICV PERC Input and Library

Generic ESD Runset File Overview



Runset Files

The following files make up the runset for checking both topology rules (netlist-based) and current density and point-to-point resistance (layout-based).

esd_core.rs

Top level PXL runset file. All calls to `perc_analyze_netlist()` are here, as well as the generation of the output Oasis layout containing the device and routing shapes involved in all discharge paths. It also contains the `perc_extract_resistance()` command which sets up and calls StarRC. This file references the following files:

esd_core_init.rs	- initialization code
esd_violation_defs.rs	- rule descriptions

Runset Writer Changes

- Any changes of the default commands for StarRC
- Any changes to the list of layers included in the discharge path Oasis file

[esd_core_init.rs](#)

PXL file which performs the following:

- Setup defaults for user inputs
- Setup device models for netlist import
- Sets up and calls [perc_setup\(\)](#) to create PERC netlist database
- Run a parse check on Python modules at startup

This file references the following files:

esd_devices.rs	- all device models needed for recognizing ESD protection are described
netlist extract runset	- for layout-based checks, the netlist extraction runset is included

Runset Writer Changes

- No changes likely

[esd_devices.rs](#)

PXL file listing every device model involved in recognizing ESD protection structures.

Runset Writer Changes

- It is critical for the runset writer to make sure all device models used in ESD protection scheme recognition are described here!

[esd_violation_defs.rs](#)

PXL file which lists all rules checked in the runset including a description and any check values.

Runset Writer Changes

- All rules, their descriptions, and any checks values must be included here.

[esd_topology_models.py](#)

Python file which contains the following:

- Definition of each type of device used as ESD protection
- Definition of device properties involved in rule checking, including how those properties are computed and combined in series and parallel
- Definition of each type of valid ESD protection, including:
 - Primary up objects
 - Primary down objects
 - Secondary series resistance
 - Secondary up objects
 - Secondary down objects

- Power clamps
- Anti-parallel diodes between grounds
- Cross-domain protection devices

Runset Writer Changes

- Must include a complete definition of all ESD protection structures.

[esd_preprocess.py](#)

Python file which performs the following:

- Recognize and tag power and ground nets
- Recognize and tag I/O pad nets
- Recognize and cache all ESD protection circuitry based on definitions defined in `esd_topology_models.py`

Runset Writer Changes

- Include the library calls to recognize each protection structure defined in `esd_topology_models.py`

[esd_rules.py](#)

Python file which performs the following:

- All code which checks/verifies topology rules
- Generates errors, including reference objects, for any ESD violations
- For current density and/or point-to-point resistance, calls code defined in `pres_rules.py` to setup check paths

Runset Writer Changes

- Setup the composites for every protection structure defined in `esd_topology_models.py`
- All code to check the rules defined in `esd_violation_defs.rs` is here

[pres_violation_defs.py](#)

Python file which contains the current density constraints and describes all discharge paths which should be checked for current density and/or point-to-point resistance.

Runset Writer Changes

- The current density constraints must be defined
- All current paths and their descriptions must be defined

[pres_rules.py](#)

Python file which contains code defining current paths that will be analyzed for current density and/or point-to-point resistance. It also contains the code which analyzes the current and voltage results after parasitic extraction to report violations.

Runset Writer Changes

- All current paths listed in `pres_violation_defs.py` must be defined

[esd_utility.py](#)

Python file which contains utility functions to support ESD checking.

Runset Writer Changes

- No changes likely

Topology Check Command Execution Flow

The below information demonstrates the flow of files and functions called by the IC Validator Generic ESD Testcase for an ESD topological check.

esd_setup.rs

- Setup information relating to the nets in the design, variables for the run, pointers to files and runsets used

esd_core.rs

- Initialize [esd_topology_models\(\)](#) (From ESD_LIBRARY) by validating [esd_preprocess.py](#) file
- Call [esd_topology_recognize\(\)](#) in [esd_preprocess.py](#)
 - Tag nets, esd devices that will be checked
 - Save netlist_db used in later functions
- Store ESD Violation Definitions [esd_violation_defs.rs](#)
 - Hash of rule names to violation values
 - Hash of rule names to violation descriptions
- Call [check_esd_topoolgy\(\)](#) in [esd_rules.py](#)

esd_rules.py

- Retrieve the [esd_topology_models](#) (Initialized earlier) – NOTE: Past this point all checking is done relative to the top cell and is not hierarchical
- Trace the topologies and populate the Tables that store the ESD Information for topologies ([trace_composite_topologies](#)). Call this for each type of ESD protection
 - IO Protection
 - Secondary
 - Power Clamps
- Check found topologies with the [process_io_esd_topology\(\)](#) function.
 - Check if valid ESD topologies exist on the net
 - No:
 - Gather data about what is on the net
 - Report error
 - Yes:
 - Check topologies against violation values
 - Number of devices in series meet rule limits
 - Device parameters meet rule limits, etc.
 - Use [ndb_report_top_cell_net\(\)](#) to report violations
 - If this is not a current density run, return to the [esd_core.rs](#) file and exit PERC checking
 - If this is a current density run continue with the below:
 - Call [create_esd_current_paths\(\)](#) function in [pres_rules.py](#)

Current Density Check Command Execution Flow

The below information demonstrates the flow of files and functions called by the IC Validator Generic ESD Testcase for an ESD Current Density (CD) and Point-to-Point (P2P) resistance check.

esd_core.py

- Perform the steps listed in the **esd_core.py** [section](#) of the [Topology Check Command Execution Flow](#)

esd_rules.py

- Called by **esd_core.py**
- Perform all the steps in the ESD Topology Checking Flow
 - These steps are in the [check_esd_topology\(\)](#) function located in [esd_rules.py](#) and called from the [esd_core.rs](#) file.
 - Steps involve identification and checking of ESD structures: Primary/Secondary protection, clamp identification, etc.
- From the [esd_rules.py](#) file call [create_esd_current_paths\(\)](#) function in [pres_rules.py](#)
 - This will use the device information which was populated in the previous step ([check_esd_topology\(\)](#) function) to inform PERC what paths to calculate current densities and point to point resistance analysis on.

pres_rules.py

- Enter the [create_esd_current_paths\(\)](#) function called from the [esd_rules.py](#) file
 - Use the [trace_*_path\(\)](#) functions to gather all nets and devices that are a part of a Current Density or Point to Point Resistance Path
 - Analyze composite tables generated in the [esd_preprocess.py](#) file
 - Create a current path by adding the below information to [ndb_add_current_path\(\)](#)
 - Sources
 - Sinks
 - Devices along the path
 - Tag of the Current Density/P2P resistance check – Note, this must match the pathtag of the violation set up in the [pres_violation_defs.py](#) file
 - Return to the [esd_core.rs](#) file after completion of the above

esd_core.rs (Return)

- Return current path information from the [perc_analyze_netlist\(\)](#) results_db option
- Call [perc_extract_resistance\(\)](#) to extract parasitics. Provide the below information:
 - Current paths (From the results_db result above)
 - Resistance information
 - STAR RC Options
- Call [check_esd_current_density\(\)](#) function located in [pres_rules.py](#)
 - Initialize reporting files
 - Retrieve current paths with [ndb_get_paths_database\(\)](#)
 - Set the current constraints and injection currents for checks (Use the ones set in [pres_violation_defs.py](#))
 - [calculate_cd_path_result\(\)](#) in [pres_rules.py](#)
 - Perform violation check with [ndb_report_current_density_error\(\)](#)
 - [calculate_p2p_path_result\(\)](#) in [pres_rules.py](#)

- Perform point-to-point resistance checking with
`ndb_report_point_topoint_error()`
 - Save extracted nets to a netlist for outputting to an OASIS file if desired
 - Return to [esd_core.rs](#)
- Create the debug OASIS file using the nets saved in the final step in the [pres_rules.py](#) `check_esd_current_density()` function.

Running IC Validator PERC & Exercises

The below exercises are intended to acquaint users with the flow of a PERC ESD deck and how to modify the sample deck for different types of checking. The exercises are intended to be performed in order as they build off each other, meaning that the same runset is modified for Exercise1, again modified for Exercise 2 and so on.

Testcase Contents

1. Copy the testcase data from the IC Validator installation into your local space
cp -rp \${ICV_HOME_DIR}/contrib/PERC/examples/esd_complete
2. You should find the following directories and files in this package.

data/

ICV_PERC_EXAMPLE.cdl – netlist for ESD protection circuit

ICV_PERC_EXAMPLE_1.cdl – modified netlist for Exercise 1

ICV_PERC_EXAMPLE_2.cdl – modified netlist for Exercise 2

ICV_PERC_EXAMPLE.oas – layout for ESD protection circuit

dummyM8process.nxtgrd – StarRC technology file

runset/

lvs.rs – runset to extract netlist from layout for current density / point-to-point resistance flow

ESD runset files

work_area/netlist_mode/

esd_setup.rs – user settings for checking netlist-only ESD rules

run – script to run IC Validator PERC

work_area/layout_mode/

esd_setup.rs – user settings for checking both netlist-based rules and current density and point-to-point resistance

run – script to run IC Validator PERC

run_esd – script called by ***run***

doc/

ESD_reference.pdf – this document

Running IC Validator PERC

Netlist-Only Checks

IC Validator PERC can run both before and after layout. Pre-layout, all the purely netlist-based rules can be checked on the schematic netlist. An example of this can be run in the **netlist_mode** directory.

- Move into the **netlist_mode** directory
- Edit the **run** script
 - Make sure IC Validator 2019.06 is installed and your environment is setup properly
- Execute the **run** script

The job should produce a file called **perc_esd_protection.txt** which describes the ESD protection circuitry found in the input netlist. Any violations are described in the **ICV_PERC_EXAMPLE.LAYOUT_ERRORS** file and can be viewed using IC Validator VUE.

All Checks Including Current Density and Point-to-point Resistance

IC Validator PERC can also run post-layout to make sure the ESD protection scheme is correct, the routing in the discharge paths is sufficient for an ESD event, and the routing resistance in the discharge paths is sufficiently small. An example of this can be run in the **layout_mode** directory.

- Move into the **layout_mode** directory
- Edit the **run_esd** script
 - Make sure IC Validator 2019.06 is installed and your environment is setup properly
 - Make sure StarRC is installed and your environment is setup properly
- Execute the **run** script

The job should produce a file called **perc_esd_protection.txt** which describes the ESD protection circuitry found in the input layout. A file called **ICV_PERC_EXAMPLE.txt** contains a summary of the current density and point-to-point resistance results. Any violations are described in the **ICV_PERC_EXAMPLE.LAYOUT_ERRORS** file and can be viewed using IC Validator VUE.

For viewing current density and point-to-point resistance errors, use a combination of ICWBEV and IC Validator VUE.

- Set the following environment variable before starting ICWBEV and ICV:
setenv VUE_OASIS_HIGHLIGHT HILITEID
- Start ICWBEV and load the Oasis file from the data directory
- In ICWBEV, start ICV VUE
- Select a current density or point-to-point resistance error
- Right click on the top line of the error in the **Diagnosis** pane and select **PERC Path Heatmap...**
- You can drop the main VUE window from your screen using the **Hide VUE** button at the bottom left of the **Highlight Path** window
- Use the **Colored by...** dropdown to select how to view the path

PERC Topology Checking Exercises

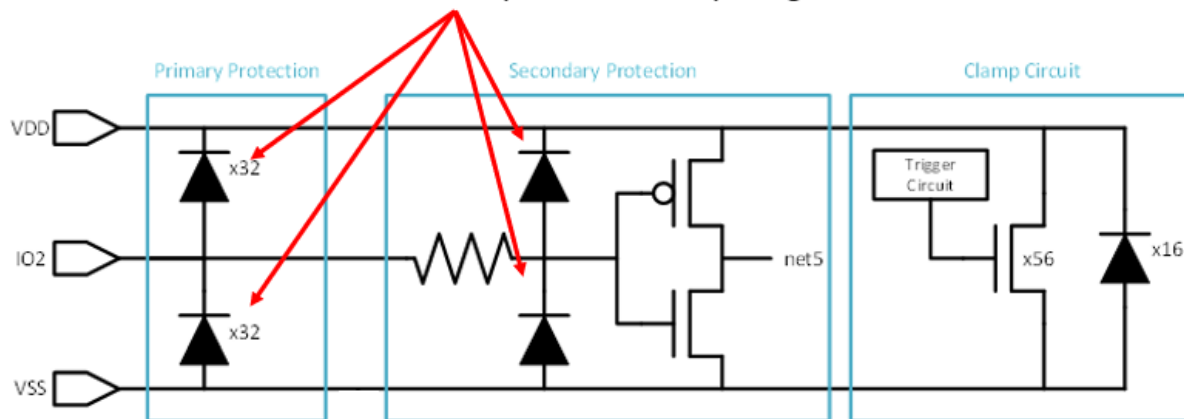
Setup

1. Move into the netlist_mode sub-directory
2. Load the latest ICV version in your environment

Exercise 1 - Defining Devices Available for ESD Protection

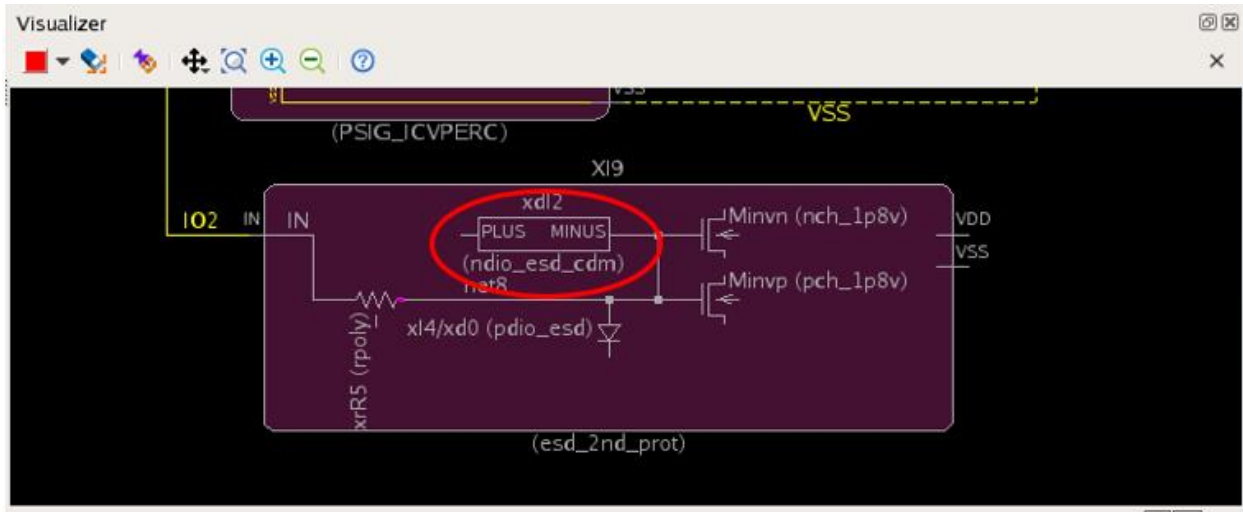
1. Edit your *esd_setup.rs* and point to the input netlist *ICV_PERC_EXAMPLE_1.cdl*
2. Rerun and check your results
 - a. What part of the protection scheme from before is now missing?
Rule ESDS5 is now being flagged, indicating that the CDM Protection devices are not being recognized. This is because the devices aren't being checked and need to be added to the runset, which we will do below.
3. Make a copy of your runset directory (i.e. **cp -rp runset runset_1**) so you will have a backup of the original runset files.
4. In the runset add the diode model ndio_esd_cdm to the set of devices for primary/secondary protection
5. Rerun (Make sure the ESD_RUNSET variable in the *run* script points to the runset directory you modified for this exercise) and make sure the full protection scheme is recognized

Add **ndio_esd_cdm** as a valid device for the different ESD protection topologies



Solution

1. By using ICV VUE to look at the secondary protection nets we can see that the “ndio_esd_cdm” device is responsible for CDM up protection



2. Open the [esd_devices.rs](#) file to add the device to the valid list of **primary_secondary_esd_diode** model names.
 - a. File is responsible for defining all the valid ESD models that can make up protection circuitry.
3. Add in the model name for the valid CDM secondary devices in the **model_category_names** hash structure.

```
model_category_names : string_string_1_h = {
  // required pin names: PLUS / MINUS
  "primary_secondary_esd_diode" => {
    "ndio_esd_cdm", // x_card=true, properties={"ndio"}
    "ndio_esd", // x_card=true, properties={"ndio"}
    "pdio_esd", // x_card=true, properties={"pdio"}
  },
  "esd_io_power_clamp_nmos" => {
    "nch_1p8v" // x_card=false
  },
  "esd_series_resistor" => {
    "rpoly" // x_card=true
  }
};
```

4. Add in the model name for the valid CDM secondary device in the **model_properties** hash. Device must be added to both hashes

```
// Device model properties
model_properties : string_string_1_h = {
  "ndio_esd_cdm" => { "x_card", "ndio" },
  "ndio_esd" => { "x_card", "ndio" },
  "pdio_esd" => { "x_card", "pdio" },
  "rpoly" => { "x_card", "lengthname=lr", "widthname=wr" }
};
```

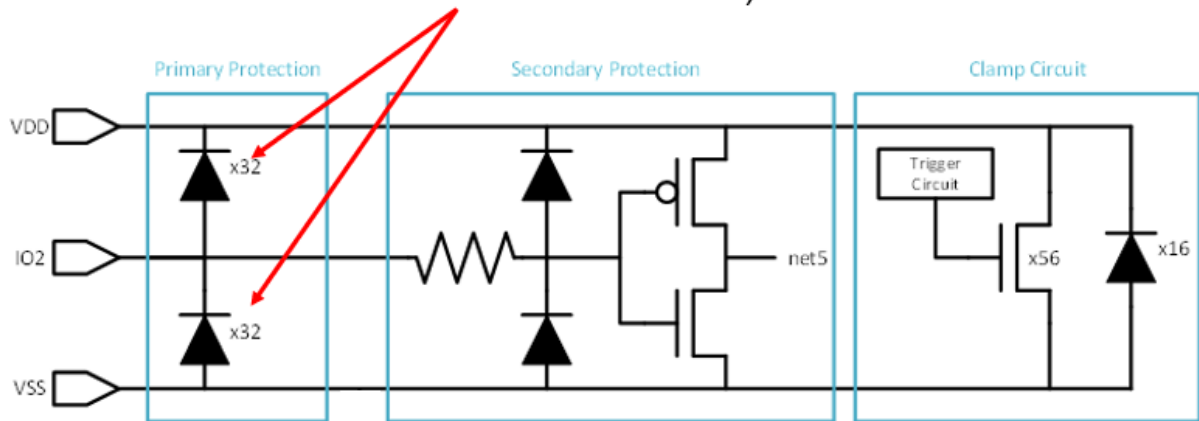
Exercise 2 - Defining ESD Structure Details

1. Edit your *esd_setup.rs* and point to the input netlist *ICV_PERC_EXAMPLE_2.cdl*
/data/ICV_PERC_EXAMPLE_2.cdl
2. Rerun and check your results – Run using the same runset you modified in [Exercise 1](#)
 - a. What part of the protection scheme from before is now missing?

The HBM up/down diode protection is now not properly recognized because there are two diodes in series in the schematic, but our topology is only configured to allow for 1 diode in series. We will update this to allow for the topology in the netlist to be recognized as a valid HBM configuration.

3. Make a copy of your runset (The same one you modified for Exercise 1) directory (i.e. **cp -rp runset_1 runset_2**) so we can make modifications on the copied runset.
4. Change the **SeriesCompositeDef** structure to recognize no more than 2 cascaded devices as a valid HBM protection structure (Currently only set to recognize no more than 1 series devices).
5. Rerun (Make sure the ESD_RUNSET variable in the *run* script points to the runset directory you modified for this exercise) and make sure the full protection scheme is recognized

Update so that no more than 2 diodes in series form a valid HBM protection structure (2 up diodes and 2 down diodes max)



Solution

The file responsible for defining valid HBM and CDM configurations is the [esd_topology_models.py](#) file.

1. Open **esd_topology_models.py** file to set up valid configurations for HBM protection.
2. Find the **SeriesCompositeDef()** structure for the primary protection
3. Find the **SeriesChainDef()** sub-structure located inside the **SeriesCompositeDef()** structure.
4. Update the **max_stage_count** value to equal 2. This variable in this structure sets the valid max number of devices in series. Example shown below (Some code removed for display purposes). Modify in total of 4 places for the HBM Up and Down diodes (Only two shown below for space).

```
# PRIMARY Composite definitions
esd_model_db.add_seriescompositedef(
    SeriesCompositeDef(
        series_composite_label = "PRIMARYUPDIO",
        report_label = "HBM ESD DIODE",
        ...
        chain_definition_list = [
            SeriesChainDef(
                connection_list = [
                    (up_ndio_primdef, dio_series_end_netdef)
                ],
                max_stage_count = 1,
                parallel_property_definition_list = [
```

```

        PropertyDef("area", prop_dio_area, ("area"))
    ],
    series_property_definition_list = [
        PropertyDef("area", prop_average_tuple, ("area"),
            ...
        )
    ],
    property_check_dict = {
        "area" : (rule_dict["ESDS1s"], "diode total area")
    }),
    SeriesChainDef(
        connection_list = [
            (up_pdio_primdef, dio_series_end_netdef)
        ],
        max_stage_count = 1,
        parallel_property_definition_list = [
            PropertyDef("area", prop_dio_area, ("area"))
        ],
        series_property_definition_list = [
            PropertyDef("area", prop_average_tuple, ("area"),
                ...
            )
        ],
        property_check_dict = {
            "area" : (rule_dict["ESDS1s"], "diode total area")
        })

```

Exercise 3 - Adding Rules / Checking Device Properties

1. Continue using the input netlist `ICV_PERC_EXAMPLE_2.cdl` referenced in your `esd_setup.rs` file.
2. Make a copy of your runset directory - The same one you modified for Exercise 2 (i.e. **cp -rp runset_2 runset_3**)
3. Add the following rule:
 - ESDS1p: An HBM up or down diode must have a minimum perimeter of 900.0 um*
 - a. Add the rule definition to `esd_violation_defs.rs`
 - b. Modify your diode `SeriesPrimitiveDef` and `SeriesCompositeDef` objects to check this property
4. Rerun (Make sure the `ESD_RUNSET` variable in the `run` script points to the runset directory you modified for this exercise) and make sure the new property rule is checked

Solution

For this case perimeter must be computed using the **l**, **w**, and **nf** parameters of the diodes in the [esd_topology_models.py](#) file. The steps to do this and check the new property are shown below.

1. Open the `esd_topology_models.py` file
2. Create a new `SeriesPrimitiveDef` property definition for the `up_ndio_primdef`, `up_pdio_primdef`, `down_ndio_primdef`, `down_pdio_primdef` structures for the perimeter as shown below (Only the `up_ndio_primdef` shown below. The modification is the same for the other primdef structures.

```

up_ndio_primdef = SeriesPrimitiveDef(
    "esd_ndio_up", JUNCTION_DIODE,
    ndio_primary_secondary_esd_diode,
    pincheck1 = ndb_pin_check(["ANODE"], ANY),
    pincheck2 = ndb_pin_check(["CATHODE"], ANY),
    property_definition_list = [

```

```

        PropertyDef("area", prop_area_prim, ("l", "w", "nf")),
        PropertyDef("perim", prop_perim_prim, ("l", "w", "nf"))
    ],
    pin1_pin2_resistance = IDEAL_DEVICE_RES
)

```

3. The above is now set to call a function called **prop_perim_prim** (Which does not yet exist). Create this new function in the file as shown below. Note this new function uses the properties of **l**, **w**, and **nf** to calculate the new **perim** value. Now the property (From Step 2) will use this function to calculate **perim**.

```

# calculate total perimeter using finger count
def prop_perim_prim(propvaltuple):
    # propvaltuple[0] -> l
    # propvaltuple[1] -> w
    # propvaltuple[2] -> nf

    try:
        lval, wval, nfval = propvaltuple

        nfval = nfval if nfval is not None else 1

        if lval is not None and wval is not None:
            return (lval * 2 + wval * 2) * nfval

        # return 'None' if area not computable for any constituent
        else:
            return None

    except ValueError:
        raise ValueError("ERROR: prop_perim_prim() found primitive " + \
            "with too many / too few property values; expected " + \
            "property count per tuple == 3. Check PropertyDef(" + \
            "input_propnames)")

```

4. Navigate to the **SeriesCompositeDef** section for the Primary HBM protection.
5. Modify the Primary **SeriesCompositeDef** structure to use the new **perim** parameter. The property needs to be added to the **SeriesChainDef** structure (sub-structure of **SeriesCompositeDef**) to account for parallel and series properties.

```

# PRIMARY Composite definitions
esd_model_db.add_seriescompositedef(
    SeriesCompositeDef(
        series_composite_label = "PRIMARYUPDIO",
        report_label = "HBM ESD DIODE",
        startnet_tagcheck = "io",
        startnetlabel = "I/O",
        endnet_tagcheck = "power",
        endnetlabel = "Power",
        tag_prefix = "dpu",
        chain_definition_list = [
            SeriesChainDef(
                connection_list = [
                    (up_ndio_primdef, dio_series_end_netdef)
                ],
                max_stage_count = 2,
                parallel_property_definition_list = [
                    PropertyDef("area", prop_dio_area, ("area")),
                    PropertyDef("perim", prop_dio_area, ("perim"))
                ],

```

```

series_property_definition_list = [
    PropertyDef("area", prop_average_tuple, ("area"),
        imbalanced_value_warning = \
        lambda propname: "cascoded devices \
        exhibit unequal finger %s values" \
        % propname,
        imbalanced_value_violation = \
        lambda propname: "cascoded devices \
        exhibit unequal finger %s values" \
        % propname
    ),
    PropertyDef("perim", prop_average_tuple, ("perim"),
        imbalanced_value_warning = \
        lambda propname: "cascoded devices \
        exhibit unequal finger %s values" \
        % propname,
        imbalanced_value_violation = \
        lambda propname: "cascoded devices \
        exhibit unequal finger %s values" \
        % propname
    )
],

```

6. Add the new **perim** property to the `property_check_dict` in the `SeriesChainDef`

```

property_check_dict = {
    "area" : (rule_dict["ESDS1s"], "diode total area"),
    "perim" : (rule_dict["ESDS1p"], "diode total perimeter")
},

```

PERC Current Density and Point-to-Point Resistance Checking Exercise

Setup

1. Navigate to the **layout_mode** sub-directory
 - Runs will take 1.20Gb memory and run for around 10 minutes
2. Load the latest ICV version in your environment
3. Edit the `run_esd` script and set the `ESD_RUNSET` environment variable to point to the original `runset` sub-directory in your location
4. Run the testcase. Text results can be viewed in the `ICV_PERC_EXAMPLE.txt` as well as graphically in ICV VUE

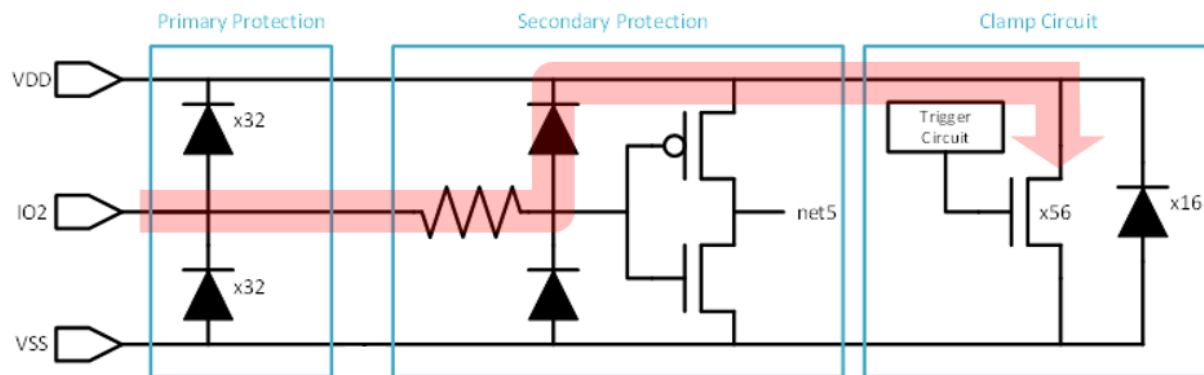
Exercise 4 - Defining Current Density and Point-to-point Resistance Paths

1. Make a copy of your runset directory - The same one you modified for [Exercise 3](#) in the PERC Topology Checking section (i.e. `cp -rp runset_3 runset_4`). You will be modifying one of these runsets, using it for runs, and saving the original for reference.
2. Edit your `run_esd` script and change the `ESD_RUNSET` environment variable to point to the runset you will be modifying for this exercise.
3. Add the following rule:

ESDP2P1: Point-to-point resistance from an I/O pad through the series resistance and the CDM up diode to the power clamp pins should be < 0.1 ohms

4. Add the rule definition to *pres_violation_defs.rs*
5. Add the path and the resistance check in *pres_rules.py*
6. Rerun and make sure the path results are checked

Add checking for the path starting at the I/O net and ending at the source of the clamp circuit MOSFET



Solution

Most of the changes for this case lie in the [pres_rules.py](#) file. The below is one such solution that is used because since our path is going from IO->secondary up diodes->Power Clamp Source we can leverage the checking done in the [trace_io_secondary_path\(\)](#) function found in the **pres_rules.py** file.

1. Open the [pres_violation_defs.py](#) file. Add path information to the pathdefs variable as shown below (Some lines for other experiments removed to save space in this document). This information will serve as the output information for the check as well as the tag used to identify the check.

```
# Current density / Point-to-point violation rule / tag /
# violation comment definitions
pathdefs = (
    ("primary_up_diodes",          ('ESDCD', 'ESDP2P1'), 'primary', \
        "I/O pad -> primary UP diode anode pin -> primary UP diode cathode pin -> Power pad"),
    ("primary_dn_diodes",          ('ESDCD', 'ESDP2P1'), 'primary', \
        "I/O pad -> primary DOWN diode cathode pin -> primary DOWN diode anode pin -> Ground pad"),
    ...,
    ("secondary_up_diodes_pc_path", (None, 'ESDP2P2'), 'secondary', \
        "I/O pad -> RESD pin1 -> RESD pin2 -> secondary UP diode anode pin -> \
        secondary UP diode cathode pin -> power clamp pin1")
)
```

2. Open the [pres_rules.py](#) file and navigate to the [trace_io_secondary_path\(\)](#) function.
 - This function uses the [ndb_add_current_path\(\)](#) command to add current density and point to point resistance checking paths. This function takes the below inputs:
 - sources – Where will we start the checking?
 - ✓ The I/O net for this example
 - sinks – Where will the checking end?
 - ✓ The Source (Power connected) pins of the clamp device for this example
 - device_paths – What devices are connected along this path?

- ✓ For this path we need the resistor tied between the IO net and the secondary protection net as well as the CDM up diodes leading to power. We will **not** need the clamp devices as the path ends at their source terminals and does not include them. For example, if this check ended at the ground net and went through the power clamp, we would need to also add the clamp fets in this list.
 - add_tags – What is the name of the “tag” associated with this check
 - ✓ This was the name that we added in the `pathdefs` variable in step 1 (“secondary_up_diodes_pc_path”).
 - shorting information between pins – How should the pins of the source and sink be treated, i.e. should source current be divided between all sources on the same net?
2. Modify the `trace_io_secondary_path` function to use information from the `powerclamp_table` structure. This is a `SeriesCompositeDef` class populated when the `esd_preprocess.py` file stage is run. Since we will be using this information as well as the list of power net names (`power_netname_list` which was provided to the `pres_rules.py` file when the `create_esd_current_paths()` function was called in `esd_rules.py`) we need to update the arguments we pass into this function to include this information.
- Once in the function it’s a good idea to make sure that the analyzed `SeriesCompositeDef` structures – the `*_table` structures aren’t empty which is the second bit of highlighted code in the function.

Function (`pres_rules.py`):

```
def trace_io_secondary_path(secondary_netname_list, resd_composite_path,
    secondary_up_dio_table, secondary_down_dio_table, powerclamp_table,
    power_netname_list, rule_dict, netcache, debugFlag = False):
```

Function Call (`pres_rules.py`):

```
trace_io_secondary_path(secondynet_netname_list, resd_composite_path,
    secondary_up_dio_table, secondary_down_dio_table, powerclamp_table,
    power_netname_list, rule_dict, netcache, debugFlag = debugFlag)
```

```
p2p_ruleobj = rule_dict["ESDP2P2"] if "ESDP2P2" in rule_dict else None
cd_ruleobj = rule_dict["ESDCD"] if "ESDCD" in rule_dict else None
```

```
if (p2p_ruleobj is None or not p2p_ruleobj.is_selected_rule()) and \
    (cd_ruleobj is None or not cd_ruleobj.is_selected_rule()):
    return
```

```
if debugFlag:
    print("trace_io_secondary_path():")
```

```
if secondary_netname_list and \
    secondary_up_dio_table is None and \
    secondary_down_dio_table is None and \
    powerclamp_table is None:
```

```
    raise ValueError("trace_io_secondary_path() " + \
        "detected \'secondary_up_dio_table is None\' and " + \
        "\'secondary_down_dio_table is None\' and " + \
        "\'powerclamp_table is None\' for non-empty " + \
```

```
"secondary_netname_list")
```

3. The entire function below is posted with changes highlighted. Note the highlighted sections at the end. What this does is leverage the `secondary_up_dio_table` checking which adds an `ndb_add_current_path()` command from IO->Resistor->Secondary Up Diodes->Power Net. Since this is already checked we just need to add another experiment that is the same as the previous but ends at the power clamp rather than the Power Net. So, for example, IO->Resistor->Secondary Up Diodes->Clamp Terminals.

```
def trace_io_secondary_path(secondary_netname_list, resd_composite_path,
secondary_up_dio_table, secondary_down_dio_table, powerclamp_table,
power_netname_list, rule_dict, netcache, debugFlag = False):

    # Store the rule information in the below variables so we can check that the rules are properly
    # defined
    p2p_ruleobj = rule_dict["ESDP2P2"] if "ESDP2P2" in rule_dict else None
    cd_ruleobj = rule_dict["ESDCD"] if "ESDCD" in rule_dict else None

    if (p2p_ruleobj is None or not p2p_ruleobj.is_selected_rule()) and \
        (cd_ruleobj is None or not cd_ruleobj.is_selected_rule()):
        return

    if debugFlag:
        print("trace_io_secondary_path():")

    # These below populated class structures are passed into the function and contain
    # information about the ESD devices (Terminals, pins, connections, etc.). See documentation
    # for information about what is stored and can be retrieved from these classes.
    if secondary_netname_list and \
        secondary_up_dio_table is None and \
        secondary_down_dio_table is None and \
        powerclamp_table is None:

        raise ValueError("trace_io_secondary_path() " + \
            "detected \'secondary_up_dio_table is None\' and " + \
            "\'secondary_down_dio_table is None\' and " + \
            "\'powerclamp_table is None\' for non-empty " + \
            "secondary_netname_list")

    if not secondary_netname_list: return

    # Make sure that the ESD resistor between the primary and secondary protection exists
    if resd_composite_path is None:
        raise ValueError("trace_io_secondary_path() " + \
            "detected \'resd_composite_path is None\' " + \
            "for non-empty secondary_netname_list")

    #####
    # Process resd_composite_path ndb_pin_terminal /
    # ndb_device_current_path objects
    #

    print("calling PathComposite.build_composite_current_path_items():")
    print(" secondary netnames: %s" % secondary_netname_list)

    # rule for requiring R-property for CDM resistors
```

```

r_ruleobj = rule_dict["ESDS4"] if "ESDS4" in rule_dict else None

# Key Concept - ICV PERC can build current paths if users pass
# in all information about paths, terminals, and nets along that path.
# In the below the resistor information on the CD/P2P path is being provided
# The device's connected nets are stored, along with the device itself
# (device_pathlist_flat = True) along with the terminals of the device as these
# will all be a part of the CD/P2P path
# build ndb_pin_terminal / ndb_device_current_path
# structures for all secondary nets in resd_composite_path
resd_composite_path.build_composite_current_path_items(
    path_ruleobj = r_ruleobj, netcache = netcache,
    reference_net_func = (lambda iterdict: [secondary_netname \
        for secondary_netname in secondary_netname_list \
        if secondary_netname in iterdict]),
    device_pathlist_flag = True,
    reference_net_pin_terminallist_flag = True,

    device_resistance_source = "property",
    debugFlag = True)
# Using the ESD Library we can grab the devices and terminals in the design that are
# part of the ESD protection. This can be used in ndb_current_path() calls for our CD/P2P checks
resd_device_pathlist_dict = \
    resd_composite_path.get_device_pathlist_dict()
resd_secnet_pin_terminallist_dict = \
    resd_composite_path.get_reference_net_pin_terminallist_dict()

# In the same way we stored the information about the resistor between CDM/HBM
# protection we will now do the same for the diodes and nets that make up the current path
# Note: This can be done in any order, we don't need to find the resistors first, or the diodes
# first, we just need to store all the information along the path and ICV PERC can figure out
# how the path was constructed in the layout.
pathdef_tuplist = [
    (secondary_up_dio_table, "secondary_up_diodes"),
    (secondary_down_dio_table, "secondary_dn_diodes")
]

# Loop over the above to store the path information for both the up and down CDM diode paths
for composite_table, pathtag in pathdef_tuplist:

    # Key Concept - ICV PERC can build current paths if users pass
    # in all information about paths, terminals, and nets along that path.
    # The below is the same storing of path information as we did above with the resistor.
    # The device's connected nets are stored, along with the devices themselves
    # as the devices are along the path (Since the path goes through the diode strings)
    # (device_pathlist_flat = True). We don't store terminals as we don't terminate the
    # CD/P2P resistance checking at any of these terminals
    # (We will terminate at the ground/power nets
    # for the up/down CDM diodes and at the clamp terminals for our new check.
    if composite_table is not None:
        composite_table.build_composite_current_path_items(
            startnet_func = (lambda iterdict: [secnetname \
                for secnetname in secondary_netname_list \
                if secnetname in iterdict]),
            device_pathlist_flag = True,
            startnet_pin_terminallist_flag = False,
            endnet_pin_terminallist_flag = False,
            debugFlag = debugFlag
        )

    # The ESD Library classes contain retrievable information
    # about the ESD device types in the design
    # The below gets the devices in the path from the table (Diodes in this case)

```

```

composite_device_pathlist_dict = \
    composite_table.get_device_pathlist_dict()

# add the RES -> secondary protection side paths
for secnetname in resd_secnet_pin_terminallist_dict:
    if debugFlag:
        print("  secondary net name: %s" % secnetname)

    # For each of the IO nets use the ndb_add_current_path() command to build up the
    # collection of start & stop points as well as devices along the path of our CD/P2P check
    for ionetname in resd_secnet_pin_terminallist_dict[secnetname]:
        ionet = netcache.get_netobj(ionetname)
        if ndb_is_net_port(ionet):
            ionet_terminal = ndb_top_port_terminal(ionet)
        else:
            continue

        resd_devices = \
            resd_device_pathlist_dict[secnetname][ionetname]

        if secnetname in composite_device_pathlist_dict:
            try:
                for endnetname, composite_device_pathlist in \
                    composite_device_pathlist_dict[secnetname].items():

                    # Actual call that stores the CD/P2P check.  Variable pathtag
                    # must match the string defined in the pathdefs variable in the
                    # pres_violation_defs.py file.
                    # Also contains all the devices along the path and informs
                    # the tool how current sources on the same
                    # net should be handled
                    ndb_add_current_path(
                        sources = [ionet_terminal],
                        sinks = netcache.get_nettermllist(endnetname),
                        device_paths = composite_device_pathlist + \
                            resd_devices,
                        add_tags = [pathtag],
                        short = [SOURCES_ON_SAME_NET, SINKS]
                    )

            except:
                raise ValueError("ERROR: trace_io_secondary_" + \
                    "path() could not read SeriesCompositeTable" + \
                    ".build_composite_current_path_items() " + \
                    "output for pathtag=%s for " % pathtag + \
                    " for secondary netname %s" % secnetname
                )

```

Added the below code in to check for the path ending at power clamp terminals

```

# The above ndb_add_current_path() command set up CD/P2P checking for the
# IO->CDM Resistor->CDM Diode->Power/Ground checks.
# The below will do the same but create a check where the end point is the
# clamp source terminals rather than power.
if secondary_up_dio_table is not None and powerclamp_table is not None:
    diode_device_pathlist_dict = \
        secondary_up_dio_table.get_device_pathlist_dict()

    # setup the top (power) side big NFET pins
    # The below is the grabs the terminal information from the power clamps.
    # Note that we don't need the devices or the ground net terminals as they are
    # not a part of the path (device_pathlist_flag = False & endnet_pin_terminallist_flag = False)
    # The startnet terminals are needed though and this is

```

```

# set to true (startnet_pin_terminallist_flag = True)
powerclamp_table.build_composite_current_path_items(
    startnet_func = (lambda iterdict: [pwrnetname \
        for pwrnetname in power_netname_list \
            if pwrnetname in iterdict]),
    device_pathlist_flag = False,
    startnet_pin_terminallist_flag = True,
    endnet_pin_terminallist_flag = False,
    debugFlag = debugFlag
)

# now get the pin dictionary
# In the above the saved terminals can now be returned as a Python dict.
# We'll use this below when setting up the
# ndb_add_current_path() check
powerclamp_pin_terminallist_dict = \
    powerclamp_table.get_startnet_pin_terminallist_dict()

# add the RES -> secondary protection side paths
for secnetname in resd_secnet_pin_terminallist_dict:
    if debugFlag:
        print(" secondary net name: %s" % secnetname)

    for ionetname in resd_secnet_pin_terminallist_dict[secnetname]:
        ionet = netcache.get_netobj(ionetname)
        if ndb_is_net_port(ionet):
            ionet_terminal = ndb_top_port_terminal(ionet)
        else:
            continue

        resd_devices = \
            resd_device_pathlist_dict[secnetname][ionetname]

        if secnetname in composite_device_pathlist_dict:
            try:
                for endnetname, diode_device_pathlist in \
                    diode_device_pathlist_dict[secnetname].items():

                    # From the above powerclamp_pin_terminallist_dict we can loop over the
                    # clamp information for clamps between our "endnetname" (Power) and
                    # "gndnetname" (Ground).
                    # This combination of keys for this structure will return a list of pins that
                    # we will use as the "sinks" in our ndb_add_current_path() check below.
                    powerclamp_pins = []
                    for gndnetname in powerclamp_pin_terminallist_dict[endnetname]:
                        powerclamp_pins += powerclamp_pin_terminallist_dict[endnetname][gndnetname]

                    # Set up our check starting at IO net->Res->CDM Up Diodes->Clamp Pins.
                    # The "add_tags" option must match the name we gave the check in
                    # the "pathdefs" variable defined in pres_violation_defs.py
                    ndb_add_current_path(
                        sources = [ionet_terminal],
                        sinks = powerclamp_pins,
                        device_paths = diode_device_pathlist + \
                            resd_devices,
                        add_tags = ["secondary_up_diodes_pc_path"],
                        short = [SOURCES_ON_SAME_NET, SINKS]
                    )

            except:
                raise ValueError("ERROR: trace_io_secondary " + \

```

```
"path() could not read SeriesCompositeTable" + \  
".build_composite_current_path_items() " + \  
"output for pathtag=%s for " % pathtag + \  
" for secondary netname %s" % secnetname  
)
```

ESD Library Classes and Functions

SeriesComposite

Class for storing cascodable primitives defined by SeriesCompositeDef class; stored primitives are represented by "primitive tuples" (primtups), which indicates the primitive object and its corresponding SeriesPrimitiveDef object's series_primitive_label.

Class Methods

print_to_report()

Formats and renders the SeriesComposite to a designated Devreport object, as part of a parent SeriesCompositeTable object rendering

Format (3-stage example):

```
WARNING: <prim instname> : <composite warning msg>
<startnet>
    <mosinst> <proplist> ||
    <mosinst> <proplist> ||
    ...
    <mosinst> <proplist>
<series_net>
    <mosinst> <proplist> ||
    <mosinst> <proplist> ||
    ...
    <mosinst> <proplist>
<series_net>
    <mosinst> <proplist> ||
    <mosinst> <proplist> ||
    ...
    <mosinst> <proplist>
<endnet>
```

Arguments

- startnetname - name of corresponding start net
- endnetname - name of corresponding end net
- supplemental_primitive_violationrule_dict - dictionary of primitive category / instance name tuples, to a list of violation rule names violated by the primitive; supplemental dict is derived during parent SeriesCompositeTable processing, and contains data only for violations processed across multiple SeriesComposite objects supplemental_primitive_violationrule_dict[("DEVICE" | "CELL", prim_instname)] -> [rulename, rulename, ...]
- protreport - Devreport class object for writing protection report with SeriesCompositeTable details
- report_line_prefix - prefix (usually spacer) added to all protreport inst section lines
- composite_count_min - minimum number of parallel primitives required to generate a composite
- debugFlag - debug flag

Returns

None

PathComposite

Class for storing generic cell-level traced paths corresponding to a particular PathCompositeDef definition; paths are stored in [node1][node2]->[list-of-primitives] 'incidence dictionaries', which are indexed according to reference-net / opposite-net names.

Dictionaries are also present to store:

- path property values
- corresponding property calculation warning / error messages for path objects
- reference net / opposite net ndb_pin_terminal objects for current path processing
- ndb_device_current_path objects for current path processing

A 'reference net' is defined as the net at the beginning of the cell-level traced path structure and all related data structures. An 'opposite net' is defined as the net at the end of the cell-level traced path structure and related data structures. A 'reference net' may correspond to either the PathCompositeDef 'start net' or 'end net' (and vice-versa for 'opposite net'), as indicated by the PathComposite class member:

```
_reference_net_category = "start" | "end"
```

Regardless, reporting / violation outputs are always formatted in 'start net' / 'end net' order.

Class Methods

trace_composite_path()

Performs cell-level path tracing and incidence_dict creation between:

- a) a designated list of reference net names, if specified; else
- b) nets matching annotated AND logged tags for shortednet + startnet (reference_net_category = "start") / endnet (reference_net_category = "end") for the corresponding PathCompositeDef model definition object, if logged tags exist for both categories; else
- c) nets matching startnet_tagcheck (reference_net_category = "start") / endnet_tagcheck (reference_net_category = "end") for the corresponding PathCompositeDef model definition object

and

- d) a designated list of opposite net names, if specified; else
- e) nets matching annotated AND logged tags for shortednet + endnet (reference_net_category = "start") / startnet (reference_net_category = "end") for the corresponding PathCompositeDef model definition object, if logged tags exist for both categories; else
- f) nets matching endnet_tagcheck (reference_net_category = "start") / startnet_tagcheck (reference_net_category = "end") for the corresponding PathCompositeDef model definition object

Traced paths are defined by PathCompositeDef parameters `_primitive_definition_dict` and `_break_path_net_tag_list`. Successfully located paths are stored in `_incidence_dict[reference_net_name][opposite_net_name]`.

A net tagged with one or more PathCompositeDef `_break_path_net_tag_list` tags prevents path propagation. If a net satisfying `opposite_netname_list` or the corresponding opposite-net tagcheck is also tagged with one or more break-path net tags, a path will be reported to the net, but will not propagate beyond the net.

Arguments

- `topcell` - flattened topcell for path tracing
- `reference_netname_list` - list of reference net names used to successfully begin cell-level path extraction and path data storage; nets may correspond to PathCompositeDef start or end nets depending on 'reference_net_category' flag value
- `opposite_netname_list` - list of opposite net names used to successfully terminate cell-level path extraction and path data storage; nets may correspond to PathCompositeDef end or start nets depending on 'reference_net_category' flag value
- `series_reference_nets` - True | False, indicating whether a path continues through one 'reference' net on its way to a different 'reference' net
- `series_opposite_nets` - True | False flag, indicating whether a path continues through one 'opposite' net on its way to a different 'opposite' net
- `debugFlag` - debug flag

Returns

None, but populates `_incidence_dict` with extracted path results per the specified reference/opposite net parameters

process_path_properties()

Performs all path-property computations for specified reference net / opposite net combinations inside the PathComposite `incidence_dict`. Function performs steps to:

- a) execute primitive PropertyDef objects for calculating primitive-level properties;
- b) execute path PathPropertyDef objects for calculating path-level properties;
- c) execute ViolationRule property checks defined by PathCompositeDef

For each specified reference-net / opposite net name, function isolates the relevant incidence sub-dictionary and invokes all PathPropertyDef `_propfunction` functions on that sub-dictionary. Output computed path properties are stored, and relevant warning/error messages are rendered.

Warning/error results are mapped back to primitive path tuple(s) responsible for those warning/error conditions.

Per-refnet/opnet path results:

- property value results: `_path_prop_dict` class object
- calculation warning: `_path_prop_warnings_dict` class object
- calculation errors: `_path_prop_errors_dict` class object

The user may provide optional iterator functions to generate selected reference-net / opposite-net values for property calculation of only selected composite groups. Without these, all reference-net / opposite-net values are iterated.

Arguments

- netcache - NetCache object to optimize net name/object retrieval
- reference_net_func - function to generate iterable of desired reference-net names to be processed; function accepts single 'iterdict' argument representing _incidence_dict
- opposite_net_func - function to generate iterable of desired opposite-net names to be processed for a given reference net; function accepts 'reference_net_name' argument representing selected reference net name, and 'iterdict' argument representing _incidence_dict[reference_net_name]
- calculate_property_flag - True | False flag indicating whether to calculate properties for specified paths; if 'False', assumption is that properties are pre-calculated and are only being checked
- check_property_flag - True | False flag indicating whether to execute ViolationRule checks on properties for specified paths; if 'False', assumption is that properties are only being calculated and may be checked later
- primcache - storage dictionary for use with is_prim_parallel(), to prevent multiple lookups of same primitive instance
- debugFlag - debug flag

Returns

None

print_to_report()

Formats and renders subpath(s) indicated by reference-net name / opposite-net name lambda functions, to a designated Devreport object; format is as follows:

Report format:

```
<path report label>: <startnet> <startnet label> -> <endnet> <endnet label>
    <path prop desc> = <val>
    <path prop desc> = <val>
    WARNING: <prim instname> : <path warning msg>
    WARNING: <prim instname> : <path warning msg>
    <startnet>
        <format_parallel_primitives() output>
    <endnet>
```

<format_parallel_primitives() output> is rendered by perc_utility.format_parallel_primitives()

Note that report entries are always rendered in start/end order, regardless of reference/opposite net assignments.

Arguments

- protoreport - Devreport class object for writing protection report with PathComposite details

- `netcache` - NetCache object to optimize net name/object retrieval
- `reference_net_func` - function to generate iterable of desired reference-net names to be processed; function accepts single 'iterdict' argument representing `_incidence_dict`
- `opposite_net_func` - function to generate iterable of desired opposite-net names to be processed for a given reference net; function accepts 'reference_net_name' argument representing selected reference net name, and 'iterdict' argument representing `_incidence_dict[reference_net_name]`
- `report_line_prefix` - prefix added to each report line after title line
- `allow_unprocessed_properties` - allows report to be generated for a PathComposite for which `process_path_properties()` has not been called; intended as a failsafe to block accidental report generation prior to property processing
- `composite_count_min` - minimum number of parallel primitives required to generate a composite
- `debugFlag` - debug flag

Returns

None

build_composite_current_path_items()

Build class member 'ndb_pin_terminal' and 'ndb_device_current_path' object containers, used as `ndb_add_current_path()` 'device_paths' / 'sources' / 'sinks' input arguments to help define logical netlist paths for parasitic resistance (current density, path resistance) analysis.

`ndb_add_current_path()` input argument object lists are built based on arguments that specify:

- the desired reference nets / opposite nets for PathComposite processing
- flags designating whether to extract
 - i. `self._reference_net_pin_terminallist_dict`: dictionary that maps path reference net / opposite net names, to a list of `ndb_pin_terminal` objects for reference-net-connected primitive pins
 - ii. `self._opposite_net_pin_terminallist_dict`: dictionary that maps path opposite net / reference net names, to a list of `ndb_pin_terminal` objects for opposite-net-connected primitive pins
 - iii. `self._device_pathlist_dict`: dictionary that maps path reference net / opposite net names, to a list of `ndb_device_current_path` objects for all primitives within the reference-net->opposite-net path

Whenever function detects that the specified data category has already been built for the specified startnets/endnets, then either:

- `assert_prepopulated_data_flag=True`: an assert is produced
- `assert_prepopulated_data_flag=False`: data regeneration is skipped

Arguments

- `path_ruleobj` - ViolationRule object representing current density / path resistance path check; property violations are generated using `path_ruleobj` for primitives whose 'r' property is not retrieved when `'device_resistance_source' == "property"`

- `netcache` - NetCache object to optimize net name/object retrieval
- `reference_net_func` - function to generate iterable of desired reference-net names to be processed; function accepts single 'iterdict' argument representing `_incidence_dict`
- `opposite_net_func` - function to generate iterable of desired opposite-net names to be processed for a given reference net; function accepts 'reference_net_name' argument representing selected reference net name, and 'iterdict' argument representing `_incidence_dict[reference_net_name]`
- `device_pathlist_flag` - True | False flag indicating whether to populate `self._device_pathlist_dict` for the specified reference net / opposite net values
- `reference_net_pin_terminallist_flag` - True | False flag indicating whether to populate `self._reference_net_pin_terminallist_dict` for the specified reference net / opposite net values
- `opposite_net_pin_terminallist_flag` - True | False flag indicating whether to populate `self._opposite_net_pin_terminallist_dict` for the specified reference net / opposite net values
- `assert_prepopulated_data_flag` - True | False flag indicating whether to issue an assert if device-pathlist or pin-terminal data has already been derived for the specified reference net / opposite net criteria; if not, then data regeneration is skipped for that specific reference net / opposite net
- `device_resistance_source` - "property" | "model" flag, indicating whether each primitive's resistance value is queried as a device property ("property") or as a SeriesPrimitiveDef model parameter ("model")
- `debugFlag` - debug flag

Returns

None

`get_device_pathlist_dict()`

Retrieves dictionary of refnetname + oppnetname to list of `ndb_device_current_path` objects, for path primitives within refnetname->oppnetname path; used to retrieve data for all refnetname->oppnetname paths for current density / path resistance logical path formation

Arguments

None

Returns

`self._device_pathlist_dict`

`get_reference_net_pin_terminallist_dict()`

Retrieves dictionary of refnetname + oppnetname to list of `ndb_pin_terminal` objects for all node1->node2 path primitives for which a) node1==reference net and b) primitive pin1 connects to reference net; used to retrieve data for all refnetname->oppnetname paths for current density / path resistance logical path formation

Arguments

None

Returns

`self._reference_net_pin_terminallist_dict`

SeriesCompositeDef

Class for storing characteristics of a parallel / series composite containing primitives defined by SeriesPrimitiveDef, and nets defined by SeriesNetDef.

Composite characteristics include:

- starting net and ending net tag-check expressions
- starting net and ending net reporting labels
- the initial counter value for the first stage of the series composite
- connection chain rules for the composite definition, including the SeriesPrimitiveDef / SeriesNetDef sub-definitions of a connection chain, the maximum number of stages in the chain, and the property merging / checking rules for the chain (defined by child SeriesChainDef object)
- parameters for the tag-value template used to generate tags for each primitive / net inside the composite

Class Methods

series_composite_recognize()

SeriesCompositeDef.series_composite_recognize() - performs hierarchical pre-recognition and tagging of specific composite topologies that are formed with series primitive chains between a specified start-net tag and end-net tag. Series primitive connection rules are described by SeriesPrimitiveDef and SeriesNetDef rules contained in the SeriesChainDef objects inside _chaindef_list.

Primitive tags are enumerated sequentially with integers, starting at the device with pin#1 connected to <self._startnet_tagcheck>. Integer counting begins at the value specified by <self._tag_position_init> and ends at <self._tag_position_init> + <SeriesChainDef._max_stage_count>.

Series chain devices are tagged as follows:

- <self._tag_prefix><posn#>_<self._tag_device_suffix> = first primitive connected to <self._startnet_tagcheck>
- <self._tag_prefix><posn#+1>_<self._tag_device_suffix> = second primitive
- <self._tag_prefix><posn#+2>_<self._tag_device_suffix> = third primitive
- ...

Series nets are tagged as follows:

- <self._tag_prefix><posn#+1>_<self._tag_net_suffix> = non-terminating series net connected to pin2 of primitive with <posn#>
- <self._tag_prefix><posn#+2>_<self._tag_net_suffix> = non-terminating series net connected to pin2 of primitive with <posn#+1>
- ...

Error primitives and error nets are tagged for certain primitive's nets which break series composite rules defined by SeriesPrimitiveDef + SeriesnetDef objects for

- a) pin swappability
- b) prohibited primitive pin connections to series nets

Error primitive are tagged as follows:

- `<self._tag_prefix><posn#>_<self._tag_error_device_suffix>` = primitive at `<posn#>`

Error nets are tagged as follows:

- `<self._tag_prefix><posn#+1>_<self._tag_error_net_suffix>` = error net connected to pin2 of primitives at `<posn#>`

Function outputs are as follows:

- A list of tag values for tagged non-error and error is returned, optionally appended to `<topology_primitive_tags>`, and optionally logged to SeriesCompositeDef member containers to store as NDB external attributes.
- A list of tag values for tagged non-error and error is returned, optionally appended to `topology_net_tags>`, and optionally logged to SeriesCompositeDef member containers to store as NDB external attributes.
- Tagged primitives and nets are optionally cached and/or with additional user-specified tags from `<add_primitive_tags>`, `<add_series_net_tags>`, `<add_error_primitive_tags>`, and `<add_error_net_tags>`

Arguments

- `nldb` - netlist database (in_out to read and save new tags)
- `topology_primitive_tags` - list of netlist topology primitive tags to which newly applied primitive tags are appended
- `topology_net_tags` - list of netlist topology net tags to which newly applied net tags are appended
- `cache` - True | False, indicating whether to cache identified nets / primitives
- `add_primitive_tags` - list of tags to annotate to series primitives, in addition to formulaically-derived tags
- `add_seriesnet_tags` - list of tags to annotate to series nets, in addition to formulaically-derived tags
- `add_error_primitive_tags` - list of tags to annotate to series error primitives, in addition to formulaically-derived tags
- `add_error_net_tags` - list of tags to annotate to series error nets, in addition to formulaically-derived tags
- `annotate_startnet` - True | False flag indicating whether to back trace successfully terminated composite topologies in order to identify/tag the confirmed top net of a series composite
- `annotate_endnet` - True | False flag indicating whether to tag the final terminating end net of a series composite with a formulaically-derived tag
- `log_tags` - True | False flag indicating whether to store all successfully attached formulaic tags to SeriesCompositeDef member containers, to write to NDB external attributes
- `debugFlag` - debug flag

Returns

Tuple of

- `found_primitive_tags` - list of non-error primitive tags annotated to series primitive topologies
- `found_error_instance_tags` - list of error primitive tags annotated to series primitive topologies
- `found_net_tags` - list of non-error / error net tags annotated to series primitive topologies

`set_startnet_tagcheck()`

Sets `_startnet_tagcheck` member of `SeriesCompositeDef` object; also sets `_composite_tag_list_of_lists` and `startnet/endnet` symmetry flag

Arguments

- `startnet_tagcheck` - tag_check to be set
- `log` - True | False flag indicating whether to log dynamic value as an NDB attribute
- `debugFlag` - debug flag

Returns

None

`get_primitivedefs()`

Retrieves a tuple of `SeriesPrimitiveDef` objects matching a tuple of `_series_primitive_label` strings, from the set of `SeriesPrimitiveDef` objects used for all `SeriesChainDef` objects inside `SeriesCompositeDef`

Arguments

- `model_labels` - tuple of `_series_primitive_label` strings corresponding to `SeriesPrimitiveDef` objects stored within all `SeriesChainDef` objects, which are requested for retrieval; a single `model_label` may also be supplied in order to retrieve a single `SeriesPrimitiveDef`
- `debugFlag` - debugFlag

Returns

Tuple of `SeriesPrimitiveDef` objects order-sorted to match input `model_labels` tuple; if `model_labels = ("*", )`, then all `SeriesPrimitiveDef` objects within `SeriesCompositeDef` are returned. If a specified `model_labels` value does not match a `SeriesPrimitiveDef` object, then 'None' is returned within the output tuple.

Note: if `model_labels` is a single label string, then the return value is a single `SeriesPrimitiveDef` object
|| None

`save_ndb_attributes()`

Creates NDB attributes for all primitive tags, net tags, error instance tags, and dynamically-set class members, for communication to downstream NDBs.

NDB attributes generated are:

- `<series_composite_label>_primitives = [<primitive_tag>, ...]`
- `<series_composite_label>_<chainidx>_<primitive_tag> = [<series_primitive_label>, <series_primitive_label>, ...]`
- `<series_composite_label>_<chainidx>_errinsts = [<errinst_tag>, <errinst_tag>, ...]`
- `<series_composite_label>_<chainidx>_nets = [<net_tag>, <net_tag>, ...]`
- `<series_composite_label>_members = [<membername>, ...]`

- <series_composite_label>_<membername> = [<memberval>]
- <series_composite_label>_logged_attr_flag = ["True"]

Arguments

- nldb - netlist database (in_out to read and save new tags)
- debugFlag - debug flag

get_chaindefs()

Returns self._chaindef_list member of SeriesCompositeDef object

Arguments

None

Returns

self._chaindef_list

PathCompositeDef

Class for storing characteristics of a traced path composite object containing path-connected primitives defined by SeriesPrimitiveDef. Path characteristics include:

- list of SeriesPrimitiveDef objects permissible and/or required in traced path
- starting net and ending net tag-check expressions
- traced path length constraints
- parameters for the tag-value template used to generate tags for each primitive / net inside the path
- list of PathPropertyDef objects that define path-specific property calculations
- dictionary of path-specific ViolationRule property checks
- reporting labels

Class Methods

path_recognize()

PathCompositeDef.path_recognize() - Performs hierarchical path tracing and net/primitive object tagging for the following object categories, depending on tag prefix/suffix template parameters explicitly defined by PathCompositeDef:

- all net/primitive objects related to path
- primitive objects within a propagated path
- primitive objects connected to a shorted start/end net
- start net objects participating in a propagated path
- end net objects participating in a propagated path
- shorted start/end net objects
- shorted start/end net objects, connected to primitive objects

Annotated tags are optionally logged as NDB attributes and added to topology_primitive_tags / topology_net_tags lists.

Arguments

- nldb - hierarchical netlist DB object

- `series_startnets` - True | False flag, indicating whether a path continues through one startnet on its way to a different startnet
- `series_endnets` - True | False flag, indicating whether a path continues through one endnet on its way to a different endnet
- `topology_primitive_tags` - list of netlist topology primitive tags to which newly applied primitive tags are appended
- `topology_net_tags` - list of netlist topology net tags to which newly applied net tags are appended
- `cache` - True | False flag, indicating whether to cache identified nets / primitives
- `log_tags` - True | False flag indicating whether to store all successfully attached formulaic tags to PathCompositeDef member containers, to write to NDB external attributes
- `debugFlag` - debug flag

Returns

Tuple of

- `found_object_taglist` - list containing `_object_annotag`, if applied to netlist
- `found_startnet_taglist` - list containing `_path_startnet_annotag`, if applied to netlist
- `found_endnet_taglist` - list containing `_path_endnet_annotag`, if applied to netlist
- `found_shortednet_taglist` - list containing `_shortednet_annotag`, if applied to netlist
- `found_shortednet_with_primitives_taglist` - list containing `_shortednet_with_primitives_annotag`, if applied to netlist
- `found_path_primitives_taglist` - list containing `_path_primitive_annotag`, if applied to netlist
- `found_shortednet_primitives_taglist` - list containing `_shortednet_primitive_annotag`, if applied to netlist

get_startnet_tagcheck()

Retrieves `_startnet_tagcheck` member of PathCompositeDef object

Arguments

None

Returns

Retrieved `_startnet_tagcheck` value

set_startnet_tagcheck()

Sets `_startnet_tagcheck` member of PathCompositeDef object; also sets `_path_taglist`

Arguments

- `startnet_tagcheck` - tag_check to be set
- `log` - True | False flag indicating whether to log dynamic value as an NDB attribute
- `debugFlag` - debug flag

Returns

None

propagate_tags()

Propagates an argument-specified list of generic tags through `ndb_device_path` objects defined by `PathCompositeDef`

Arguments

- `nldb` - netlist DB object
- `net_tags` - list of generic tags to be propagated through path defined by `ndb_device_path` objects generated from `PathCompositeDef._primitive_definition_dict`
- `debugFlag` - debug flag

Returns

None

set_endnet_tagcheck()

Sets `_endnet_tagcheck` member of `PathCompositeDef` object; also sets `_path_taglist`

Arguments

- `endnet_tagcheck` - tag_check to be set
- `log` - True | False flag indicating whether to log dynamic value as an NDB attribute
- `debugFlag` - debug flag

Returns

None

save_ndb_attributes()

Creates NDB attributes for all primitive tags, net tags, and dynamically-set class members, for communication to downstream NDBs.

NDB attributes generated are:

- `<path_definition_label>_objects = [<object_tag>, ...]`
- `<path_definition_label>_path_primitives = [<primitive_tag>, ...]`
- `<path_definition_label>_shortednet_primitives = [<primitive_tag>, ...]`
- `<path_definition_label>_path_startnets = [<net_tag>, ...]`
- `<path_definition_label>_path_endnets = [<net_tag>, ...]`
- `<path_definition_label>_shortednets = [<net_tag>, ...]`
- `<path_definition_label>_shortednets_with_primitives = [<net_tag>, ...]`
- `<path_definition_label>_members = [<membername>, ...]`
- `<path_definition_label>_<membername> = [<memberval>]`
- `<path_definition_label>_logged_attr_flag = ["True"]`

Arguments

- `nldb` - netlist database (in_out to read and save new tags)
- `debugFlag` - debug flag

Returns

None

TopologyDefDB

Class for storing SeriesCompositeDef objects, PathCompositeDef objects, and related SeriesPrimitiveDef objects for general bookkeeping during netlist database processing and analysis; class also handles the storage and retrieval of dynamically-set class members, netlist object tags, etc., using netlist DB (ndb) attributes.

Class Methods

add_seriescompositedef()

TopologyDefDB.add_seriescompositedef() - add SeriesCompositeDef object to the TopologyDefDB composite storage dictionary; also add associated SeriesPrimitiveDef objects linked to the composite to the TopologyDefDB primitive storage dictionary. An assert is produced if a SeriesCompositeDef object different from the input argument is already stored in the dictionary under the same series_composite_label, or if the input argument's tag definitions collide with those of an existing pre-registered SeriesCompositeDef.

Arguments:

compositedefobj - SeriesCompositeDef object to be added to DB dictionary

Returns:

None

add_pathcompositedef()

TopologyDefDB.add_pathcompositedef() - add PathCompositeDef object to the TopologyDefDB composite storage dictionary; also add associated SeriesPrimitiveDef objects linked to the composite to the TopologyDefDB primitive storage dictionary. An assert is produced if a PathCompositeDef object different from the input argument is already stored in the dictionary under the same path_definition_label, or if the input argument's tag definitions collide with those of an existing pre-registered PathCompositeDef.

Arguments:

pathdefobj - PathCompositeDef object to be added to DB dictionary

Returns:

None

does_seriescompositedef_exist()

TopologyDefDB.does_seriescompositedef_exist() - returns flag, to indicate whether the provided model_label exists as a composite label on a SeriesCompositeDef object inside TopologyDefDB.

Arguments:

model_label - unique string matching the composite label for a SeriesCompositeDef object

Returns:

'True' if model_label matches a series composite label for a SeriesCompositeDef object inside TopologyDefDB

'False' otherwise

get_seriescompositedef()

Retrieves SeriesCompositeDef object from TopologyDefDB, based on provided model_label checked against _series_composite_label for each composite. An assert is produced if no SeriesCompositeDef matching model_label is found.

Arguments

model_label - unique string matching _series_composite_label for SeriesCompositeDef object to be retrieved

Returns

SeriesCompositeDef object matching model_label

SeriesCompositeTable

Class for storing both successfully and unsuccessfully completed SeriesComposite objects corresponding to a particular SeriesCompositeDef definition; composites are stored in dictionaries according to startnet name / endnet name / SeriesChainDef definition.

Dictionaries are also present to store

- composite property values
- property error messages for parallel composite objects
- startnet / endnet ndb_pin_terminal objects for current path processing
- ndb_device_current_path objects for current path processing

Class Methods

get_composite_dict()

Retrieves three-dimensional dictionary of valid SeriesComposite objects

Arguments

None

Returns

self._composite_dict

get_composite_def()

Retrieves SeriesCompositeDef object corresponding to this SeriesCompositeTable

Arguments

None

Returns

self._composite_def

get_parallel_composite_prop_warnings_dict()

Retrieves dictionary of startnetname + endnetname + chainidx value + property-related warning message, to list of primitive tuples corresponding to warning message

Arguments

None

Returns

`self._parallel_composite_prop_warnings_dict`

get_composite_table_property_dict()

Intelligently retrieves parallel-merged composite-table property values for parallel composites at specified startnet/endnet/chainidx values, while also searching for unmerged properties from individual startnet/endnet/ chainidx composites not requiring parallel merging.

Function intelligently searches for properties for sequences with length=1 that do not require a particular stage of merging, i.e.:

- a) a single SeriesComposite for a given startnet/endnet/chainidx value -- parallel composite merging not required
- b) a single stage within a single SeriesComposite -- series merging not required [accomplished by `SeriesComposite.get_composite_properties()`]
- c) a single parallel primitive within b) -- parallel primitive merging not required [accomplished by `SeriesComposite.get_composite_properties()`]

If an unmerged property is successfully found for a single composite, the found property is stored as a composite table property in `_parallel_composite_prop_dict`.

```
self._parallel_composite_prop_dict[startnetname][endnetname][chainidx][propname] =  
propval
```

An assert is generated if the specified startnet + endnet + chainidx values are not found for SeriesComposite objects within this SeriesCompositeTable.

Arguments

- `startnetname` - name of startnet to which composite objects correspond for which property values are retrieved
- `endnetname` - name of endnet to which composite objects correspond for which property values are retrieved
- `chainidx` - SeriesChainDef index to which composite objects correspond for which property values are retrieved
- `propnames` - single composite property name, or iterable of property names, being queried; if none specified dictionary is returned containing only pre-queried properties.

Returns

`self._parallel_composite_prop_dict[startnetname][endnetname][chainidx]`, after intelligent property processing for length==1 sequences

format_error_composite_output()

Produces a violation comment stringlist, a list of cell-typed instances, a list of device-typed instances, and a list of composite endnet names, representing data necessary to generate a PYDB violation for all `_err_composite_dict` composites satisfying given start/end net names

Arguments

- `startnetname` - name of net object at top of error SeriesComposite
- `netcache` - NetCache object to optimize net name/object retrieval

- `endnetnames` - iterable or scalar of error SeriesComposite-terminating net names searched in `_err_composite_dict`; default=None, meaning all endnets are used
- `debugFlag` - debug flag

Returns

Tuple of

- `commentlist` - Violation comment string list of form:
"reason1 (<instlist>;
reason2 (<instlist>; ... "
- `nonerr_cellinst_list` - list of non-error-related cell instances inside all error SeriesComposite objects satisfying the provided start/end net names; instances are not linked to a specific error message causing the topology failure
- `nonerr_devinst_list` - list of non-error-related device instances inside all error SeriesComposite objects satisfying the provided start/end net names; instances are not linked to a specific error message causing the topology failure
- `err_cellinst_list` - list of error-related cell instances inside all error SeriesComposite objects satisfying the provided start/end net names; instances are directly responsible for topology errors
- `err_devinst_list` - list of error-related device instances inside all error SeriesComposite objects satisfying the provided start/end net names; instances are directly responsible for topology errors
- `matched_endnets` - list of endnet objects from `endnetnames` which exist as keys of `_err_composite_dict[startnetname]`; `endnet=""` is not included

format_valid_composite_output()

Produces a list of valid cell-typed instances, a list of device-typed instances, and a list of composite endnet names, representing data necessary to generate reference objects for a PYDB violation for all `_composite_dict` composites satisfying given start/end net names.

Arguments

- `startnetname` - name of net object at top of error SeriesComposite
- `netcache` - NetCache object to optimize net name/object retrieval
- `endnetnames` - iterable or scalar of SeriesComposite-terminating net objects searched in `_composite_dict`; default=None, meaning all endnets are used
- `chainidxs` - iterable or scalar of SeriesChainDef index values searched in `_composite_dict`; default=None, meaning all chain index values are used
- `debugFlag` - debug flag

Returns

Tuple of

- `cellinst_list` - list of cell instances inside all valid SeriesComposite objects satisfying the provided start/end net names and type criteria
- `devinst_list` - list of device instances inside all valid SeriesComposite objects satisfying the provided start/end net names and type criteria

- `matched_endnets` - list of endnet objects from endnetnames which exist as keys of `_composite_dict[startnetname]`;

`process_compositetable_properties()`

Performs all inter-composite parallel property merging for parallel SeriesComposite objects having the same startnetname / endnetname / SeriesChainDef index values.

Function performs steps to:

- a) execute primitive PropertyDef objects for calculating parallel composite properties; note that the same SeriesChainDef-stored PropertyDef objects define parallel merging for both parallel same-stage primitives, and parallel SeriesCompositeTable composites
- b) execute SeriesCompositeTable ViolationRule property checks defined in SeriesChainDef

For each startnetname / endnetname / chainidx value, function builds an input property value structure and passes it to `merge_property_sequence()` for computation and warning/error rendering.

Warning/error results are mapped back to primitive tuple(s) responsible for those warning/error conditions.

Parallel-merged per-startnet/endnet/chainidx composite table results:

- property value results: `_parallel_composite_prop_dict` class object
- calculation warning: `_parallel_composite_prop_warnings_dict` class object
- calculation errors: `_parallel_composite_prop_errors_dict` class object

The user may provide optional iterator functions to generate selected startnet/endnet/chainidx values for property calculation of only selected composite groups. Without these, all startnet/endnet/chainidx values are iterated

Arguments

- `netcache` - NetCache object to optimize net name/object retrieval
- `startnet_func` - function to generate iterable of desired start nets to be processed; function accepts single 'iterdict' argument representing `_composite_dict`
- `endnet_func` - function to generate iterable of desired end nets to be processed for a given start net; function accepts 'startnet' argument representing selected start net, and 'iterdict' argument representing `_composite_dict[startnet]`
- `chainidx_func` - function to generate iterable of desired `_composite_def.get_chaindefs()` SeriesChainDef index values to be processed for a given end net; function accepts 'endnet' argument representing selected end net, and 'iterdict' argument representing `_composite_dict[startnet][endnet]`
- `debugFlag` - debug flag

Returns

None

generate_primitives()

Returns generator for iterative tuples of all primitive objects and corresponding SeriesPrimitiveDef series_primitive_label values, within SeriesComposites defined by the optionally specified startnet / endnet / chainidx values inside self._composite_dict

Only valid primitives are returned from _composite_dict; error primitives inside valid composites are excluded.

Arguments

- startnetname - name of startnet to which composite objects correspond for which primitives are retrieved; if not specified, primitives for all startnet names are retrieved
- endnetname - name of endnet to which composite objects correspond for which primitives are retrieved; if not specified, primitives for all endnet names are retrieved
- chainidx - SeriesChainDef index value to which composite objects correspond for which primitives are retrieved; if not specified, primitives for all chainidx values are retrieved

Yields

primtuple – a SeriesComposite (primitive, primdeflabel) tuple

annotate_primitive_violationrule()

Add a new primitive-->rulename mapping to _parallel_composite_primitive_violationrule_dict, representing a rulename violated by a composite primitive for a composite adjoining startnetname/endnetname.

Arguments

- startnetname - name of startnet for Series Composite to which primitive_instname belongs
- endnetname - name of endnet for Series Composite to which primitive_instname belongs
- primitive_category - "DEVICE" | "CELL", indicating whether primitive is a device or cell instance
- primitive_instname - instance name of primitive for which a rulename violation entry is being recorded
- violated_rulename - string representing ViolationRule rulename, for a rule violated by primitive_instname

Returns

None

print_compositetable_to_report()

Formats and renders the list of same-startnet / same-endnet / same-chainidx SeriesComposite objects to a designated Devreport object; format is as follows:

Report format:

```
WARNING [Chain Type #n]: <prim instname> : <composite-list warning msg>
WARNING [Chain Type #n]: <prim instname> : <composite-list warning msg>
WARNING [Chain Type #m]: <prim instname> : <composite-list warning msg>
```

```

<composite report label>: <startnet> <startnet label> -> <endnet> <endnet label>
    <composite prop desc> [Chain Type #n] = <val>
    <composite prop desc> [Chain Type #m] = <val>
    [Chain Type #n composites]
    WARNING: <prim instname> : <composite warning msg>
    <composite desc>
    .....
    WARNING: <prim instname> : <composite warning msg>
    <composite desc>
    -----
    [Chain Type #m composites]
    WARNING: <prim instname> : <composite warning msg>
    <composite desc>
    .....
    WARNING: <prim instname> : <composite warning msg>
    <composite desc>

```

Note: all labels pertaining to chain type # are printed only if a particular report entry for a start->end pairing lists composites of multiple chain types

<composite desc> is rendered by SeriesComposite.print_to_report()

Arguments

- protoreport - Devreport class object for writing protection report with SeriesCompositeTable details
- startnet_func - function to generate iterable of desired start nets to be processed; function accepts single 'iterdict' argument representing _composite_dict
- endnet_func - function to generate iterable of desired end nets to be processed for a given start net; function accepts 'startnet' argument representing selected start net, and 'iterdict' argument representing _composite_dict[startnet]
- chainidx_func - function to generate iterable of desired _composite_def.get_chaindefs() SeriesChainDef index values to be processed for a given end net; function accepts 'endnet' argument representing selected end net, and 'iterdict' argument representing _composite_dict[startnet][endnet]
- report_line_prefix - prefix added to each report line after title line
- allow_unprocessed_properties - allows report to be generated for a SeriesCompositeTable for which process_compositetable_properties() has not been called; intended as a failsafe to block accidental report generation prior to property processing
- debugFlag - debug flag

Returns

None

build_composite_current_path_items()

Build class member 'ndb_pin_terminal' and 'ndb_device_current_path' object containers, used as ndb_add_current_path('device_paths' / 'sources' / 'sinks' input arguments to help define logical netlist paths for parasitic resistance (current density, path resistance) analysis.

`ndb_add_current_path()` input argument object lists are built based on arguments that specify:

- the desired startnets / endnets / chainidx values for SeriesComposite processing
- flags designating whether to extract
 - i. `self._startnet_pin_terminallist_dict`: dictionary that maps composite startnet/endnet names, to a list of `ndb_pin_terminal` objects for startnet-connected primitive pins
 - ii. `self._endnet_pin_terminallist_dict`: dictionary that maps composite endnet/startnet names, to a list of `ndb_pin_terminal` objects for endnet-connected primitive pins
 - iii. `self._device_pathlist_dict`: dictionary that maps composite startnet/endnet names, to a list of `ndb_device_current_path` objects for all primitives within the SeriesComposite

Whenever function detects that the specified data category has already been built for the specified startnets/endnets, then either:

- `assert_prepopulated_data_flag=True`: an assert is produced
- `assert_prepopulated_data_flag=False`: data regeneration is skipped

Arguments

- `startnet_func` - function to generate iterable of desired start nets to be processed; function accepts single 'iterdict' argument representing `_composite_dict`
- `endnet_func` - function to generate iterable of desired end nets to be processed for a given start net; function accepts 'startnet' argument representing selected start net, and 'iterdict' argument representing `_composite_dict[startnet]`
- `chainidx_func` - function to generate iterable of desired `_composite_def.get_chaindefs()` SeriesChainDef index values to be processed for a given end net; function accepts 'endnet' argument representing selected end net, and 'iterdict' argument representing `_composite_dict[startnet][endnet]`
- `device_pathlist_flag` - True | False flag indicating whether to populate `self._device_pathlist_dict` for the specified startnet/endnet/chainidx values
- `startnet_pin_terminallist_flag` - True | False flag indicating whether to populate `self._startnet_pin_terminallist_dict` for the specified startnet/endnet/chainidx values
- `endnet_pin_terminallist_flag` - True | False flag indicating whether to populate `self._endnet_pin_terminallist_dict` for the specified startnet/endnet/chainidx values
- `assert_prepopulated_data_flag` - True | False flag indicating whether to issue an assert if device-pathlist or pin-terminal data has already been derived for the specified startnet/endnet/chainidx criteria; if not, then data regeneration is skipped for that specific startnet/endnet/chainidx
- `debugFlag` - debug flag

Returns

None

`get_device_pathlist_dict()`

Retrieves dictionary of startnetname + endnetname to list of `ndb_device_current_path` objects, for all primitives within startnetname->endnetname SeriesComposite objects; used for current density / path resistance logical path formation

Arguments

None

Returns

self._device_pathlist_dict

General Library Functions

trace_composite_topologies()

Trace and populate SeriesCompositeTable objects containing composites connected to a given list of nets, according to a provided dictionary of SeriesCompositeDef definitions; when multiple SeriesCompositeDef objects are processed for a given net, resulting SeriesComposite objects are analyzed to post-process composite objects having common primitives across different SeriesCompositeDef definitions.

Arguments

- netname_list - list of net name strings serving as the top net of searched composite topologies
- compositedef_dict - dictionary of SeriesCompositeDef objects, for which SeriesComposite objects are traced and SeriesCompositeTable objects populated:
compositedef_dict[_series_composite_label] -> <SeriesCompositeDef obj>
- netcache - NetCache object to optimize net name/object retrieval
- primcache - primitive cache dictionary for parallel device detection
- propcache - property cache dictionary for device property handling
- ignore_startnet_primitive_list - list of primitives to ignore when processing start-nets in netname_list, for purposes of SeriesCompositeTable tracing
- configFlags - dictionary of miscellaneous configuration flags passed in by ICV top-level run configuration
- debugFlag - debug flag

Returns

compositetable_dict - dictionary of SeriesCompositeTable objects created by tracing input SeriesCompositeDef objects connected to nets in provided netname_list:

compositetable_dict[_series_composite_label] -> <SeriesCompositeTable obj>

ESD Rules and Diagrams

Please see subsequent pages for ESD Rules and diagrams for rules that are coded into the Generic ESD Runset files.

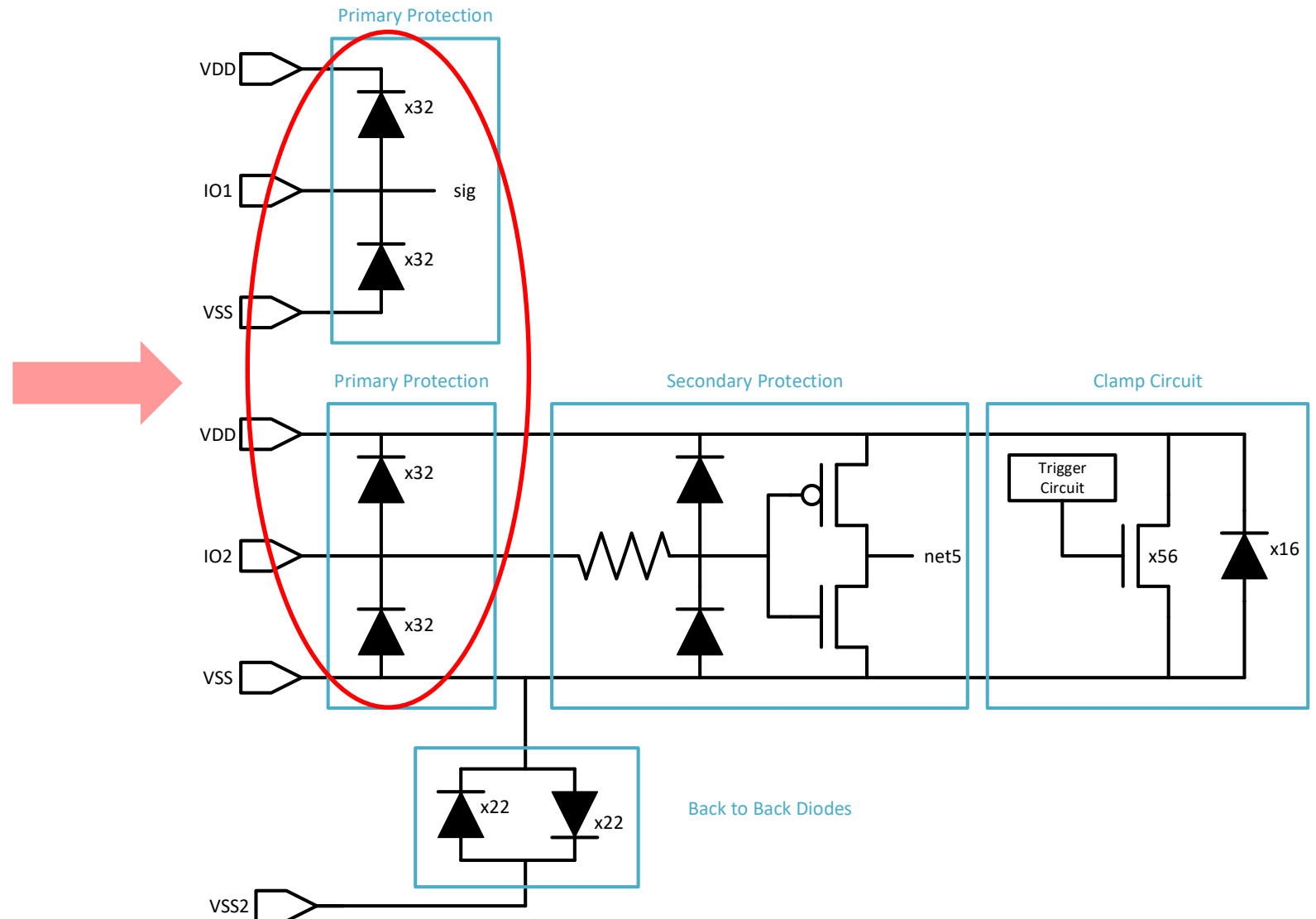
ESDS1 / ESDS1s

ESDS1:

All I/O signal pads must be connected to one or more HBM down diodes satisfying rule ESDS1s and one or more HBM up diodes satisfying rule ESDS1s

ESDS1s:

An HBM up or down diode must have a minimum area of 240 μm^2



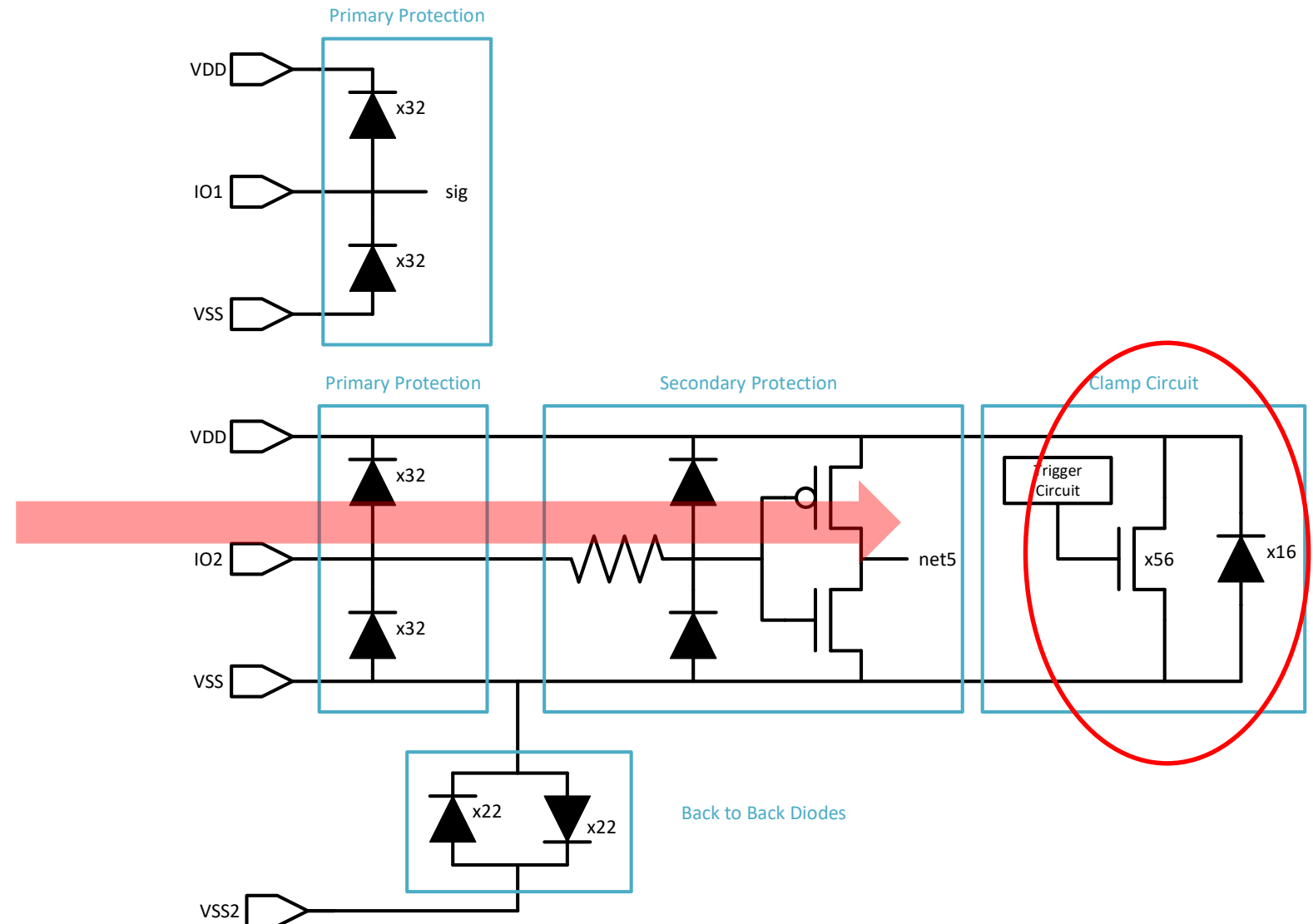
ESDS2 / ESDS2s

ESDS2:

All power supply pads must be connected to an RC-triggered power clamp satisfying rule ESDS2s and a single HBM down diode satisfying rule ESDS1s

ESDS2s:

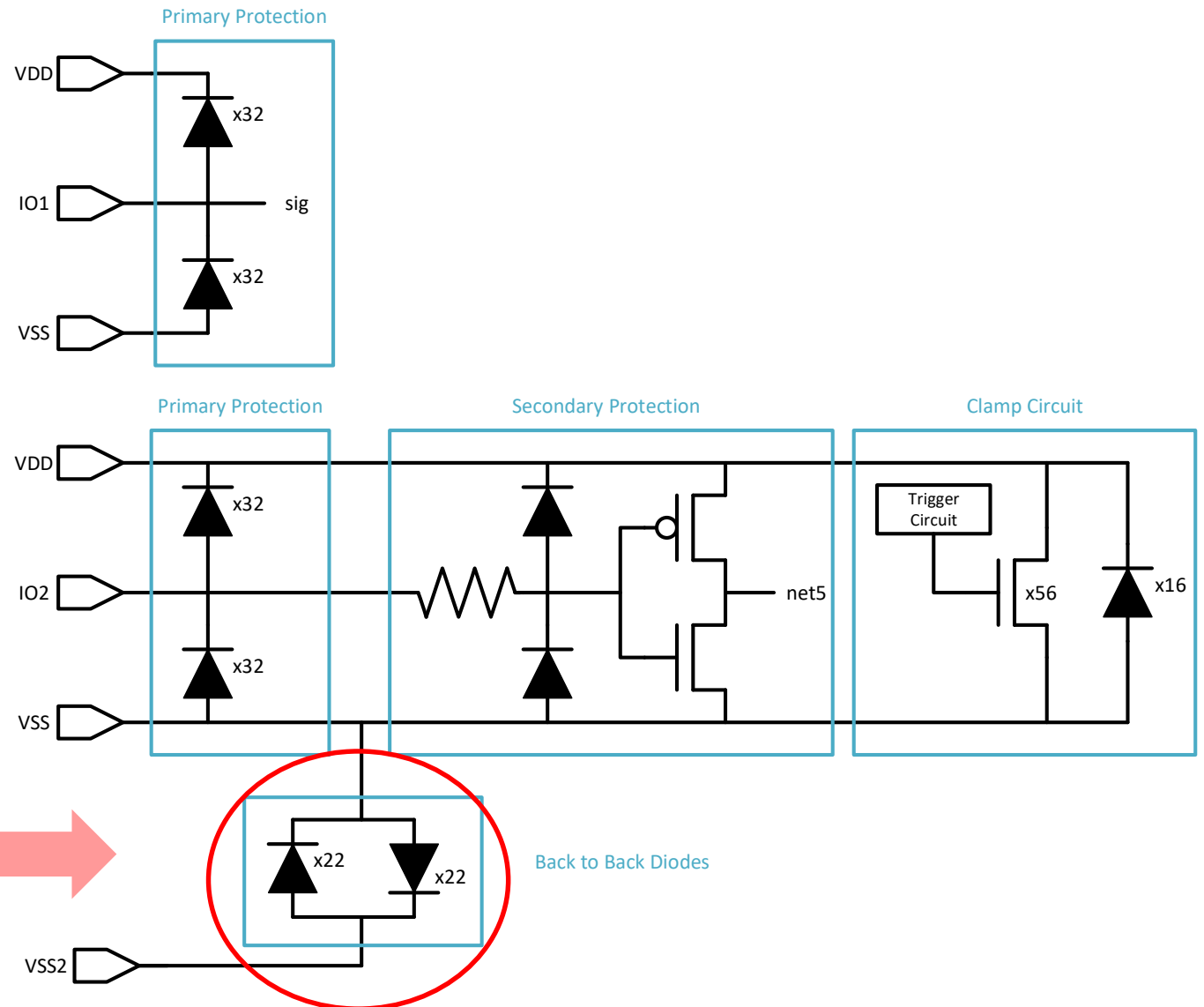
An HBM up or down diode must have a minimum area of 240 μm^2



ESDS3

ESDS3:

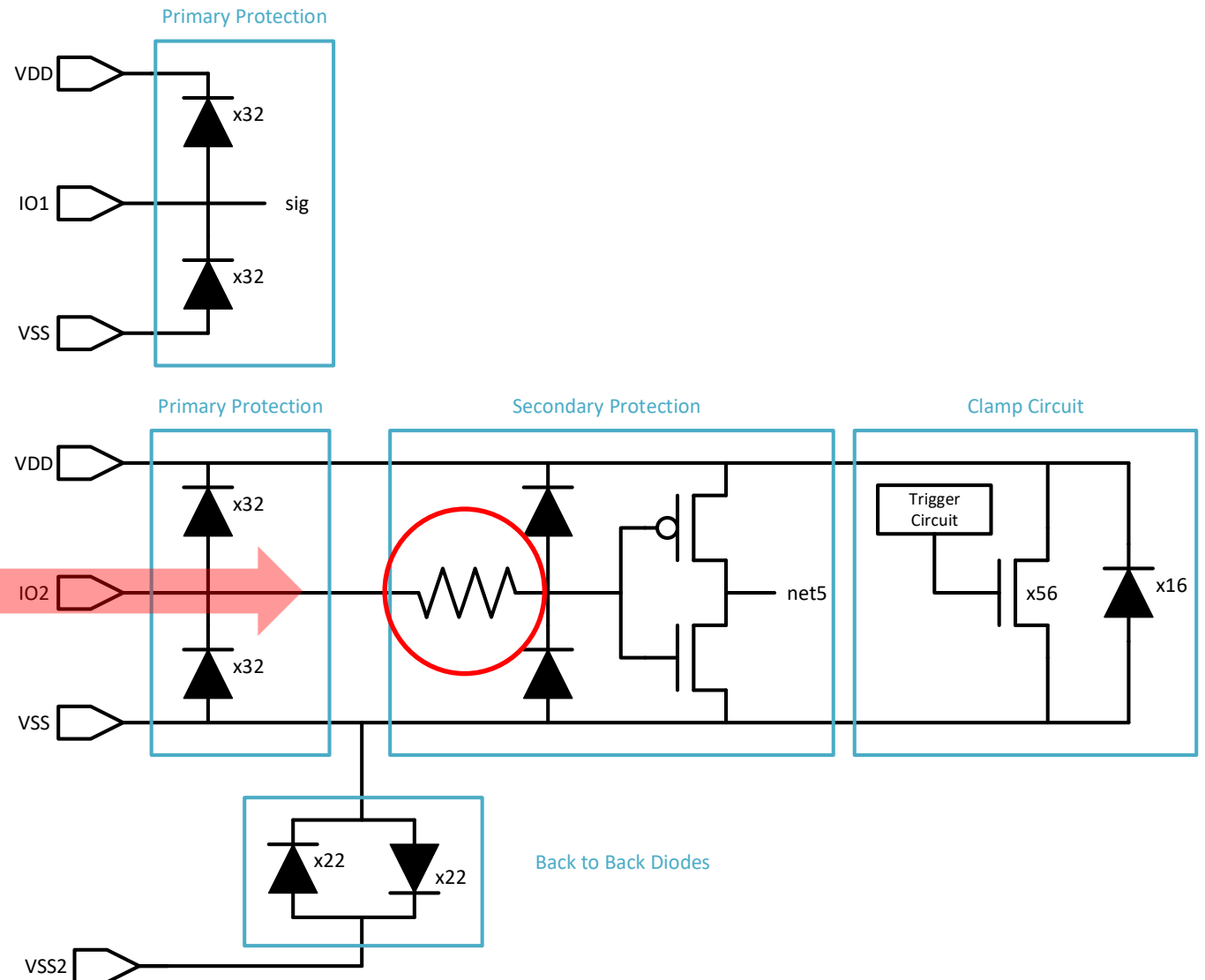
The main ground must be connected to every other ground pad using back-to-back diodes satisfying rule ESDS1s



ESDS4

ESDS4:

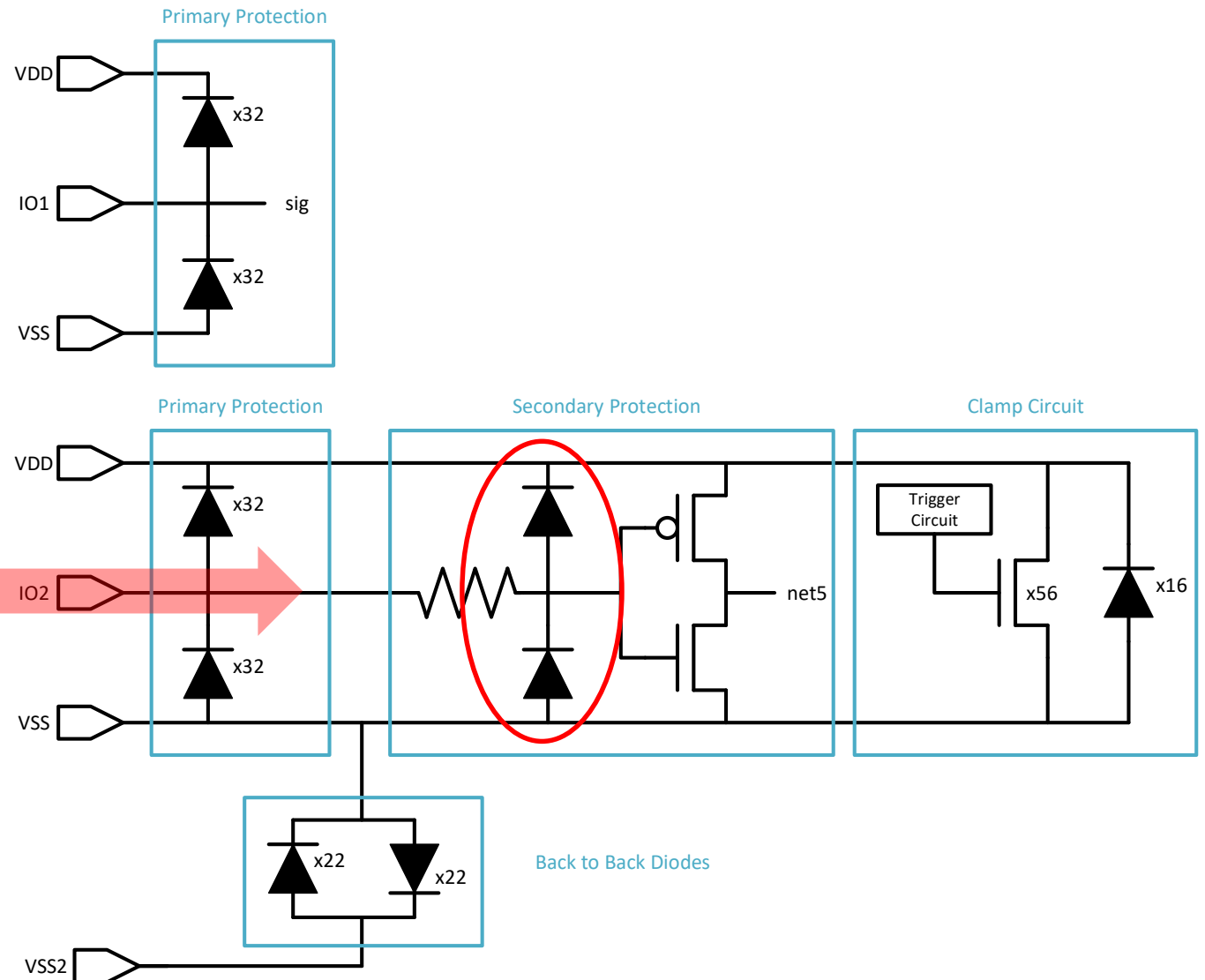
All input gates connected to an I/O signal pad should be connected through a CDM resistor



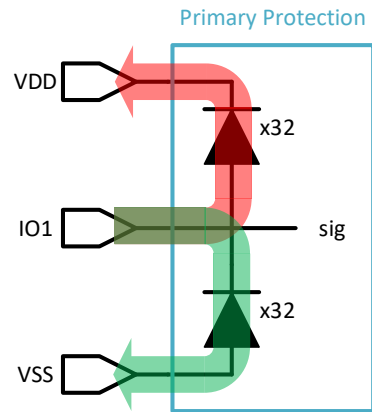
ESDS5

ESDS5:

All input gates connected to an I/O signal pad should be connected to CDM protection devices

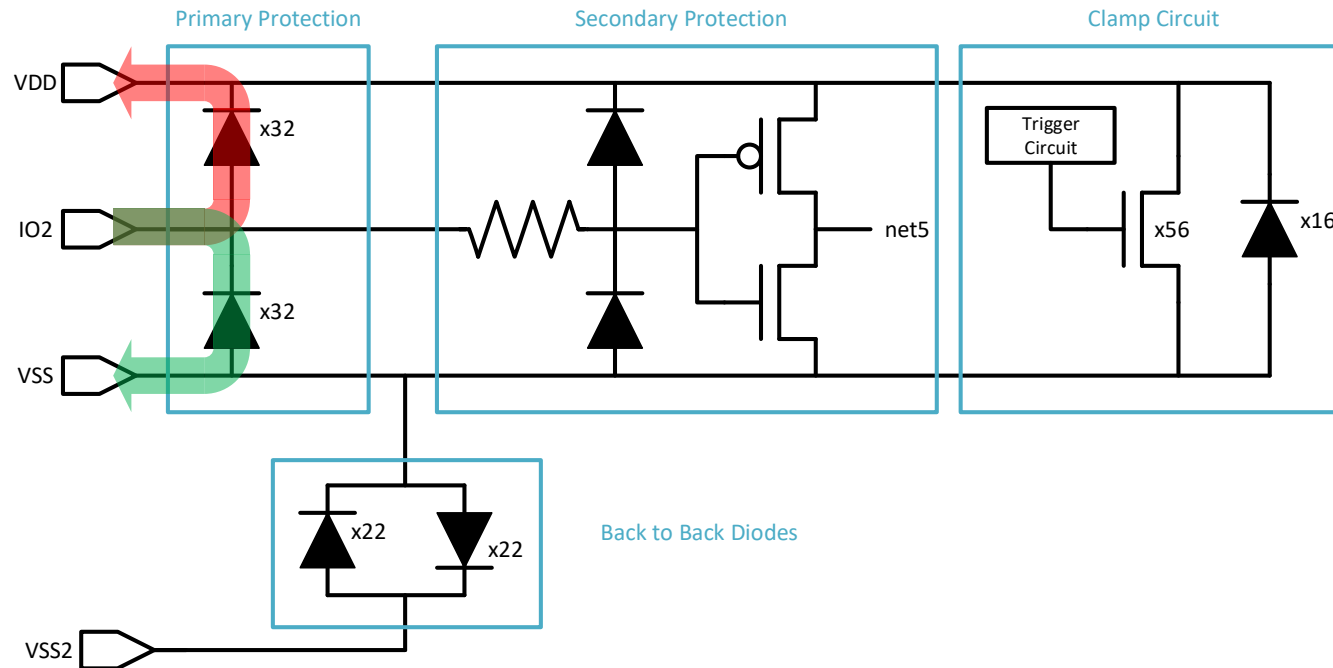


ESDCD / ESDP2P1

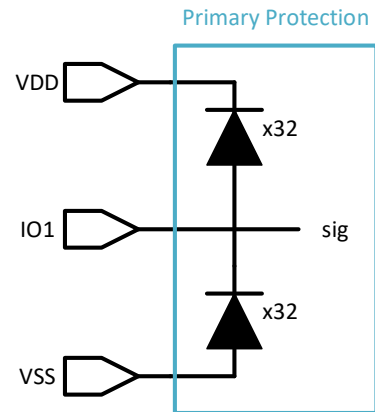


Check current density and report point-to-point resistance:

- From each I/O through the primary up protection diode(s) to the corresponding power pad
- From each I/O through the primary down protection diode(s) to the corresponding ground pad

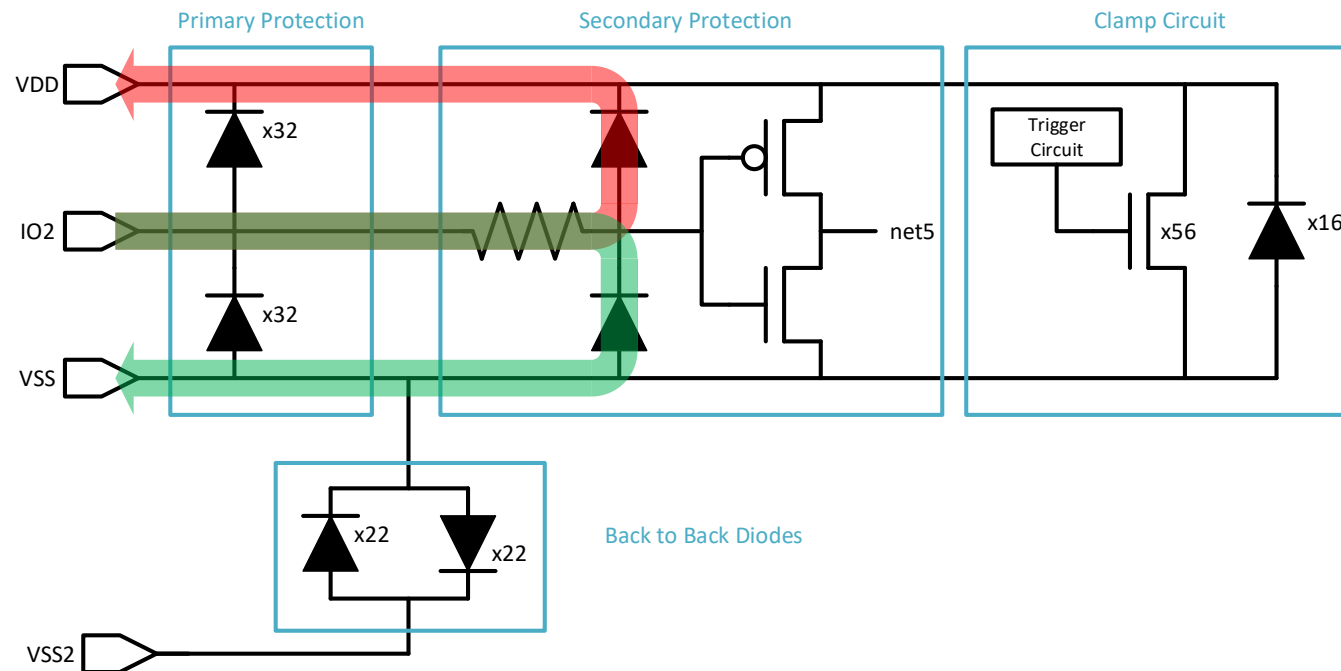


ESDCD / ESDP2P2

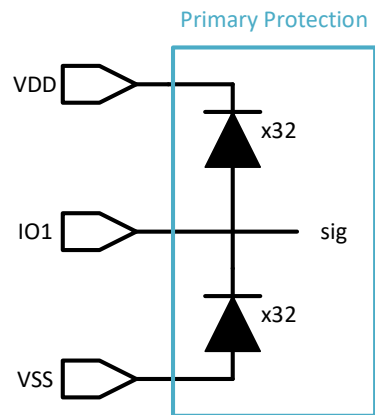


Check current density and report point-to-point resistance:

- From each I/O through the secondary series resistance and up protection diode(s) to the corresponding power pad
- From each I/O through the secondary series resistance and down protection diode(s) to the corresponding ground pad

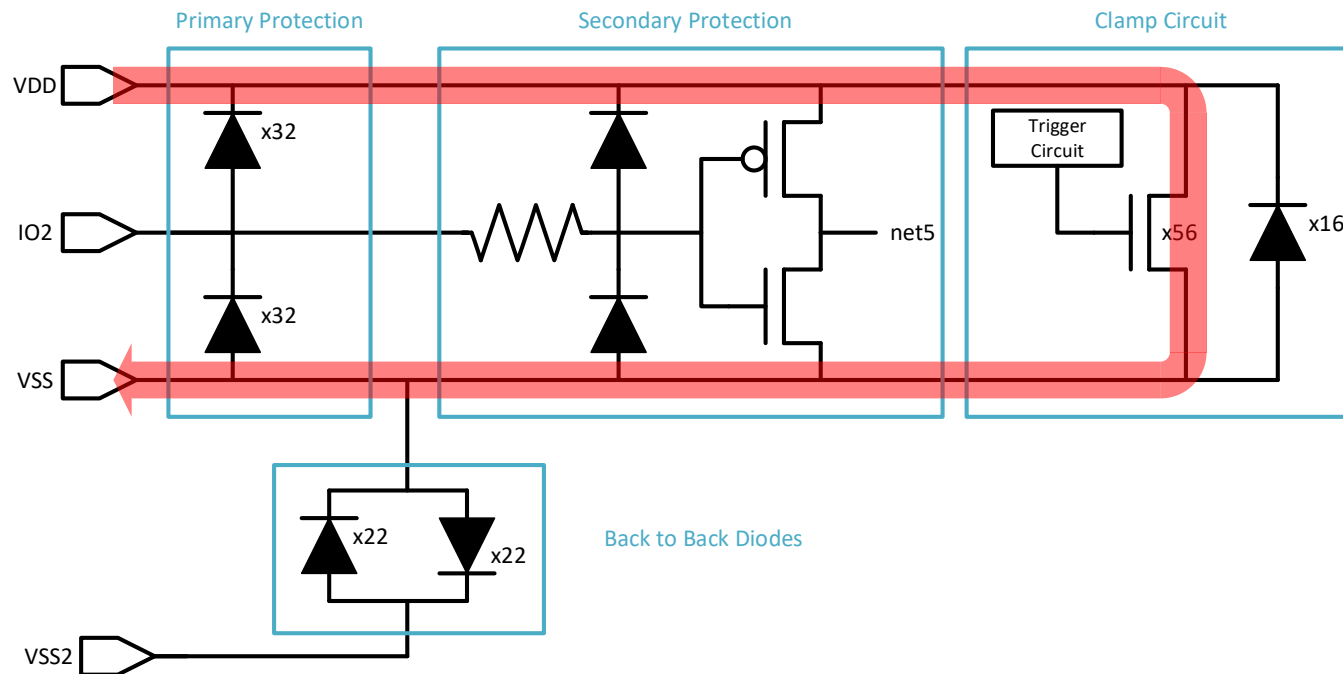


ESDCD / ESDP2P1

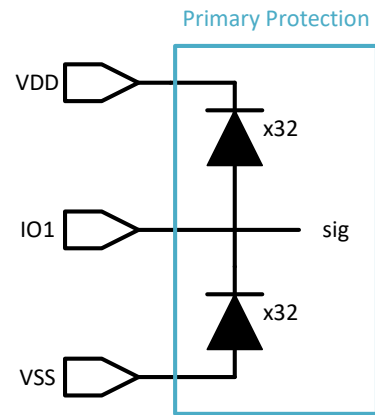


Check current density and report point-to-point resistance:

- For each nfet power clamp, from the corresponding power pad through the clamp source/drain to the corresponding ground pad

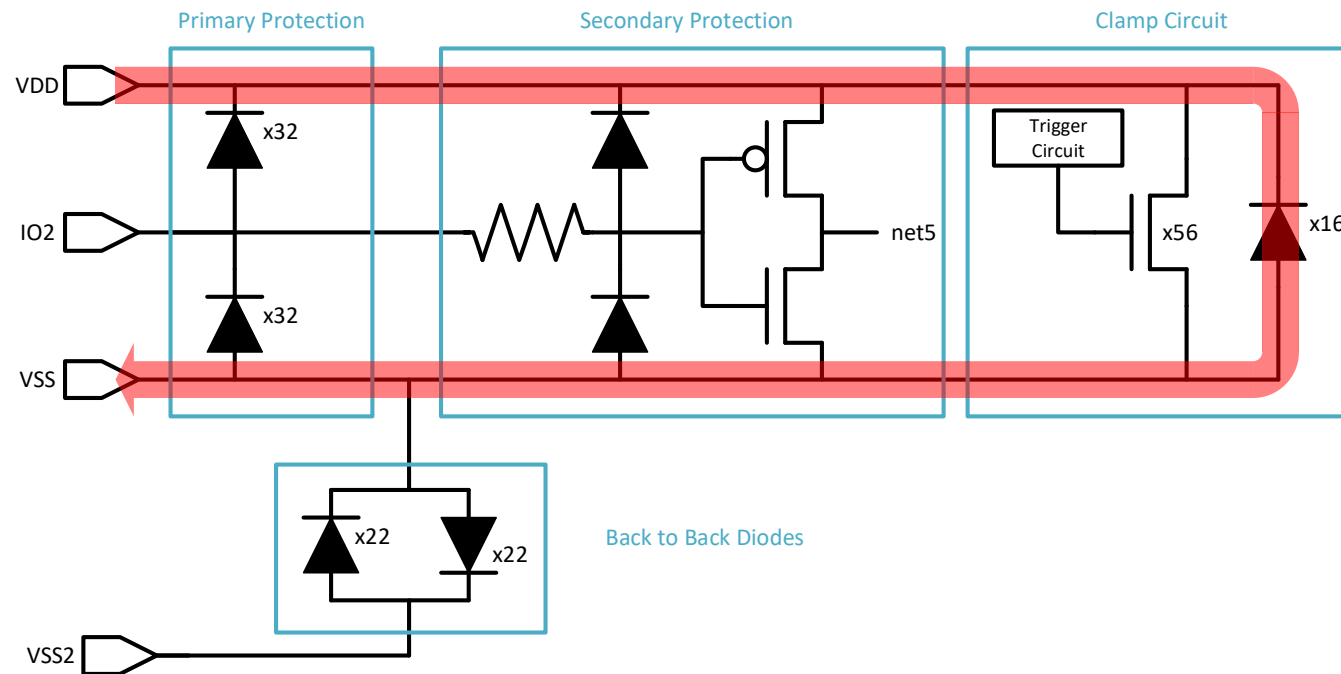


ESDCD

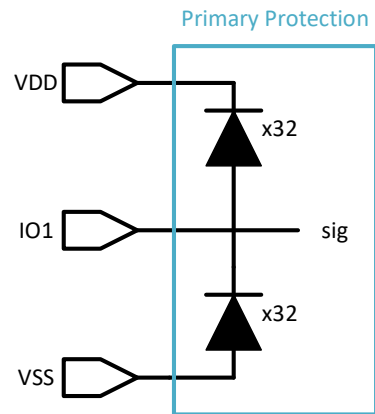


Check current density:

- For each power clamp reverse diode, from the corresponding power pad through the diode to the corresponding ground pad



ESDP2P1



Report point-to-point resistance:

- For each pair of main ground to other ground
 - From the main ground pad to the diode pins
 - From the other ground pad to the diode pins

Check current density:

- For each pair of main ground to other ground
 - From each ground pad to diode cathode pins

